

Q-ROVER 3차시

공통 미션 과제

직접 학습한 물체를 따라가는 로버



OPENING 3차시 목차

01	미션·준비	목표·일정·역할·장비 p.3
02	촬영·라벨링	STEP 1~2 데이터셋 만들기 p.12
03	학습·ONNX	STEP 3~4 추가 학습·노트북 검출 p.24
04	보드로 배포	STEP 5 모델 변환·설정·실행·측정 p.46
05	P제어·구현·튜닝	STEP 6~8 명세·부호·Kp 비교 p.66
06	시연·제출·발표	STEP 9 영상·README·팀 계획 p.100

최종 미션의 목표

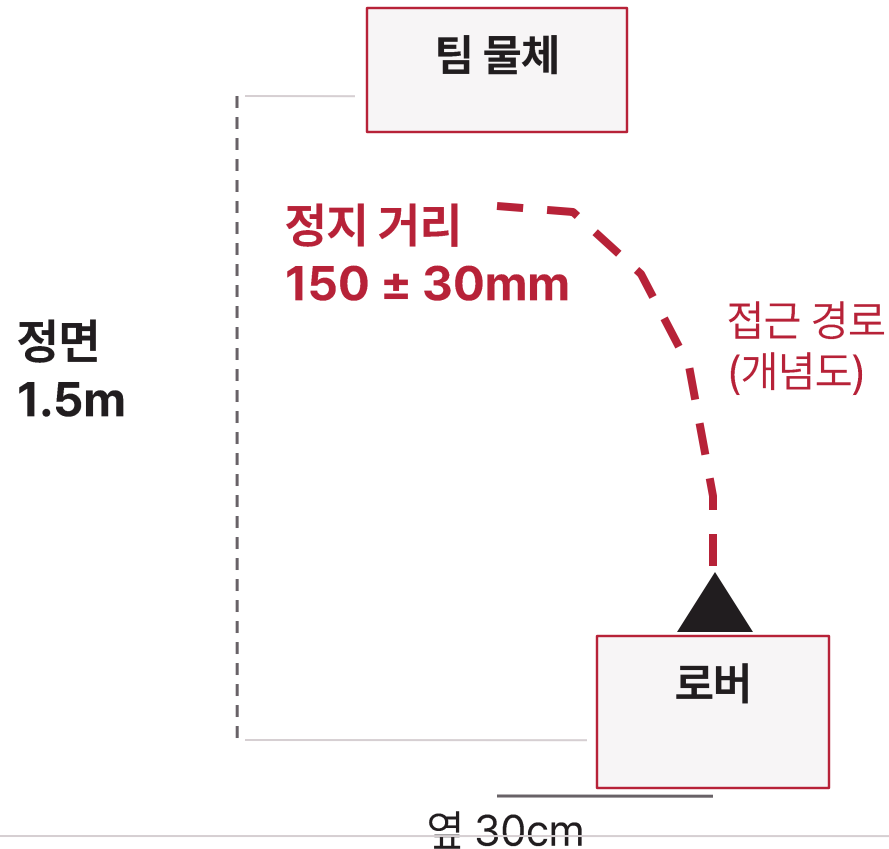
우리 팀 물체를 찾고, 가운데로 맞추며 접근

- 1.COCO에 없는 물체 1종을 촬영·라벨링하고 YOLOX-Nano를 추가 학습합니다.
- 2.로버가 물체를 카메라 화면 가운데에 맞추며 다가갑니다. ToF 센서로 거리를 확인해 물체 약 15cm 앞에서 멈춥니다.

STEP 1~5 학습·배포 / STEP 6~8 P제어 / STEP 9 시연

11/1(일)까지 실습 완료, 11/2(월) 공통 미션 결과와 팀 프로젝트 계획 발표

미션 배치와 정지 기준



시작 후 3초 대기

목표를 보면 중심을 맞추며 전진합니다.
목표가 안 보이는 동안은 정지합니다.

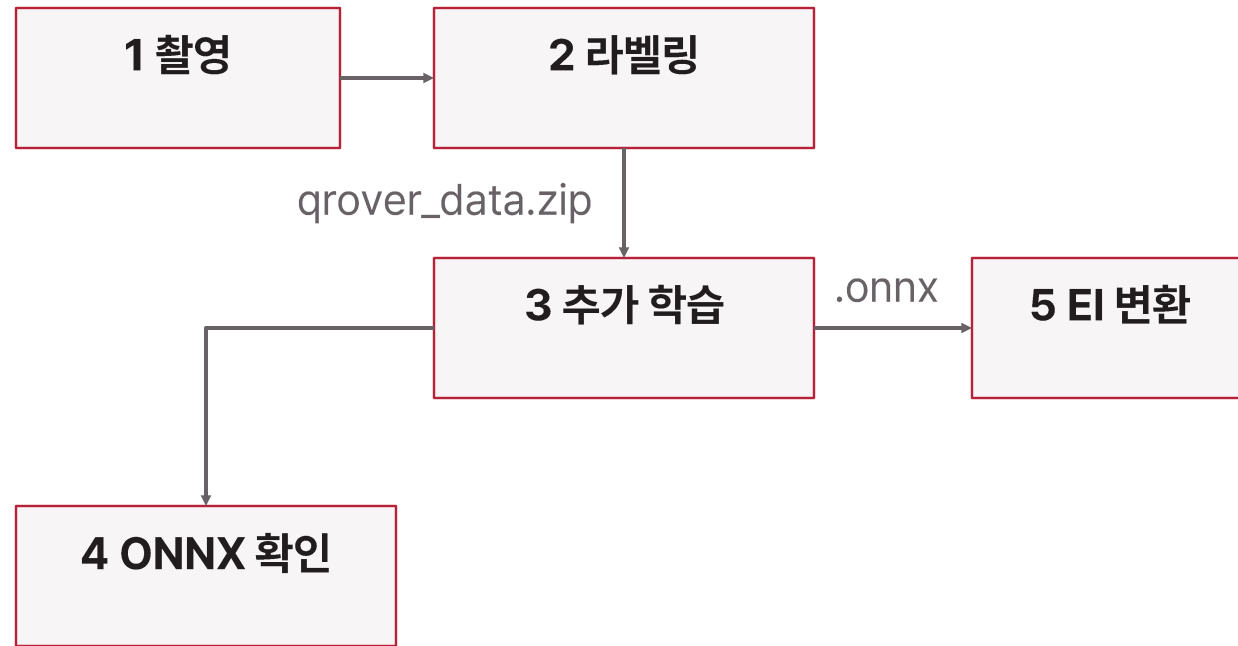
안전장치

전방 80mm 미만 장애물은 즉시 정지.
Python 명령이 1초 넘게 끊겨도 정지.

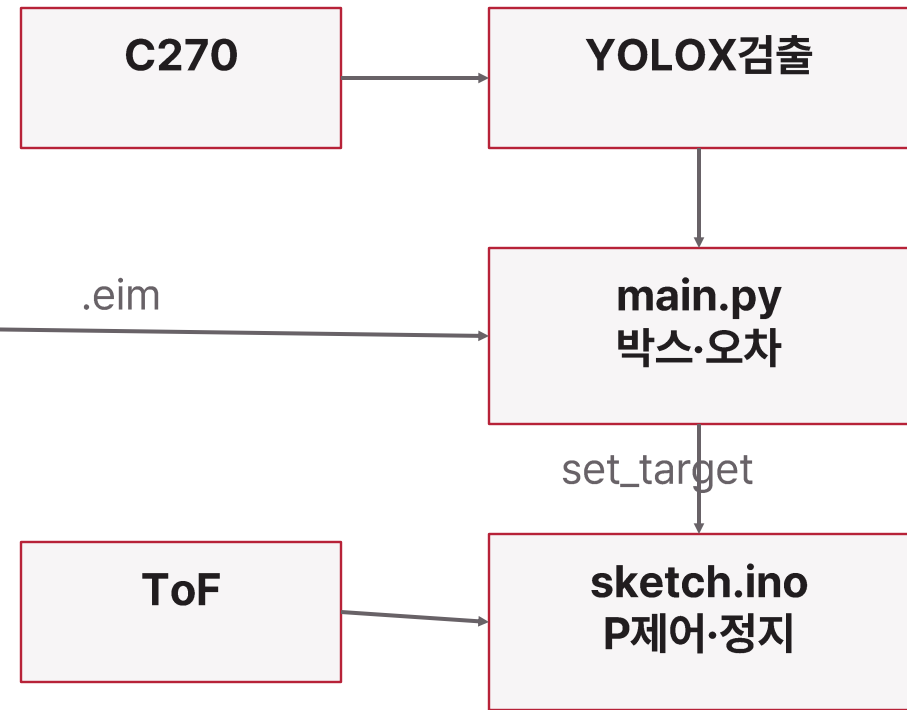
출발 시 목표가 카메라 시야에 보이는지 확인하고, 목표 물체 방향으로 로버의 각도를 조절합니다. 이후 로버가 물체 방향으로 이동하면 물체 15cm 앞에서 정지합니다.

전체 흐름과 파일

노트북·Colab



UNO Q



모터 / 로그 / 시연

권장 진행표

작업	확인·산출물
STEP 1 촬영	세션별 장수·조건
STEP 2 라벨링	라벨링 규칙·데이터셋
STEP 3 학습 / STEP 4 노트북 확인	AP50·ONNX·검출 화면
STEP 5 보드 배포·측정	FRAME_W·콜백 FPS·임계값
STEP 6 개념 / STEP 7 구현	바퀴를 들고 부호·정지 확인
STEP 8 튜닝 / STEP 9 시연	Kp 그래프·1분 영상
제출·11/2 발표 준비	팀 프로젝트 계획도 함께 정리

공통 미션과 팀 프로젝트 조사를 병행합니다.

역할

역할 (5인 기준)	주 담당	팀 규칙
데이터 리드	STEP 1·2	촬영·라벨링은 전원 분담
학습 리드	STEP 3·4	Colab 실행·학습 결과 기록
보드 리드	STEP 5·7 실행	App Lab 실행·배포 버튼 담당
제어 리드	STEP 6~8	두 명이 짝으로 부호 검증
기록·발표 리드	측정·영상·계획 발표	실패 원인과 측정표 정리

학습·배포가 막히면 기본 COCO 모델의 bottle로 STEP 6~8을 진행하고, 준비되면 팀 라벨로 바꿉니다.

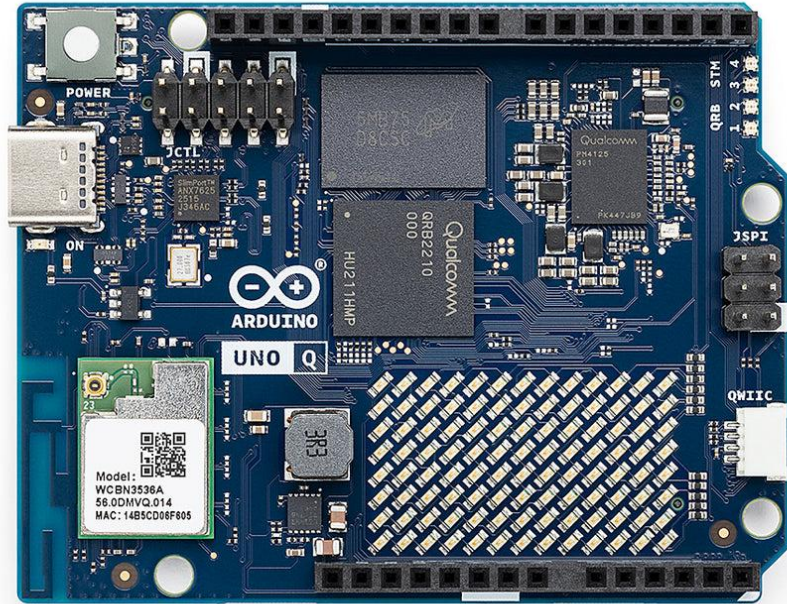
여유가 되는 팀은, 팀원 각각 다른 종류의 물체를 촬영/라벨링/학습 과정 전반을 진행하도록 합니다. -> 5인 팀원이면 총 5개의 물체 추가학습

계정과 실습 파일

준비	확인할 내용
Google	Colab·Drive. 팀 공유 폴더 권장
Edge Impulse / Arduino	두 서비스는 같은 이메일 로 가입·로그인
Arduino App Lab	0.5.0 이상. 모델 연동 메뉴 확인
노트북 Python	Python 3.10 이상. OpenCV·ONNX Runtime·NumPy
실습 묶음·코드	배포 ZIP을 풀고 STEP별 폴더 사용 https://github.com/emjdp/hyper-q-rover

노트북 터미널에 설치: `pip install opencv-python onnxruntime numpy`

장비와 주행 전원



Arduino UNO Q

Linux MPU와 MCU를 함께 사용



C270 + VL53L0X + 전원 허브

주행용 5V 3A 보조배터리를 로버에 싣습니다.
허브 전원을 먼저 넣고, 보드에 연결합니다.

팀 물체 1종과 라벨

물체 조건	확인 이유
COCO 80종에 없는 물체	컵·병·공·teddy bear는 이미 기본 클래스에 포함/ 직접 확인해보기
손바닥~A5 크기, 바닥에 세울 수 있음	원거리 검출과 ToF가 맞힐 면 확보
색·무늬가 뚜렷함	배경과 구분
투명·거울 재질을 피함	ToF 측정이 불안정할 수 있음
카메라 높이 근처 이상	가까워졌을 때 화면·ToF 시야에서 빠지지 않도록

라벨링 시, labels.txt에 hyper_mascot처럼 영문 소문자 라벨 1개를 씁니다. 대소문자·공백·기호까지 동일해야 합니다.
라벨링: 지정 물체(학습시키고자 하는 물체)에 이름표를 달아주는 과정, yolo는 이 이름표로 물체를 인식합니다.

이번 자료는 공통 미션의 실습 순서와 확인 방법을 중심으로 구성했습니다.

실습 중 필요한 개념은 스스로 찾아 학습하고, 코드 실행과 실험 결과로 이해한 내용을 검증해 보길 권합니다.

예를 들어 YOLO 모델의 학습 방식, P제어의 원리, ONNX 모델을 보드 실행용 .eim 형식으로 변환하는 이유를 살펴볼 수 있습니다.

Part 1

데이터: 촬영과 라벨링

STEP 1 사전 설정: 카메라 USB 이동

카메라 USB만 이동



카메라 USB를 노트북에 꽂아 촬영한 뒤, 촬영이 끝나면 보드에 다시 연결합니다.

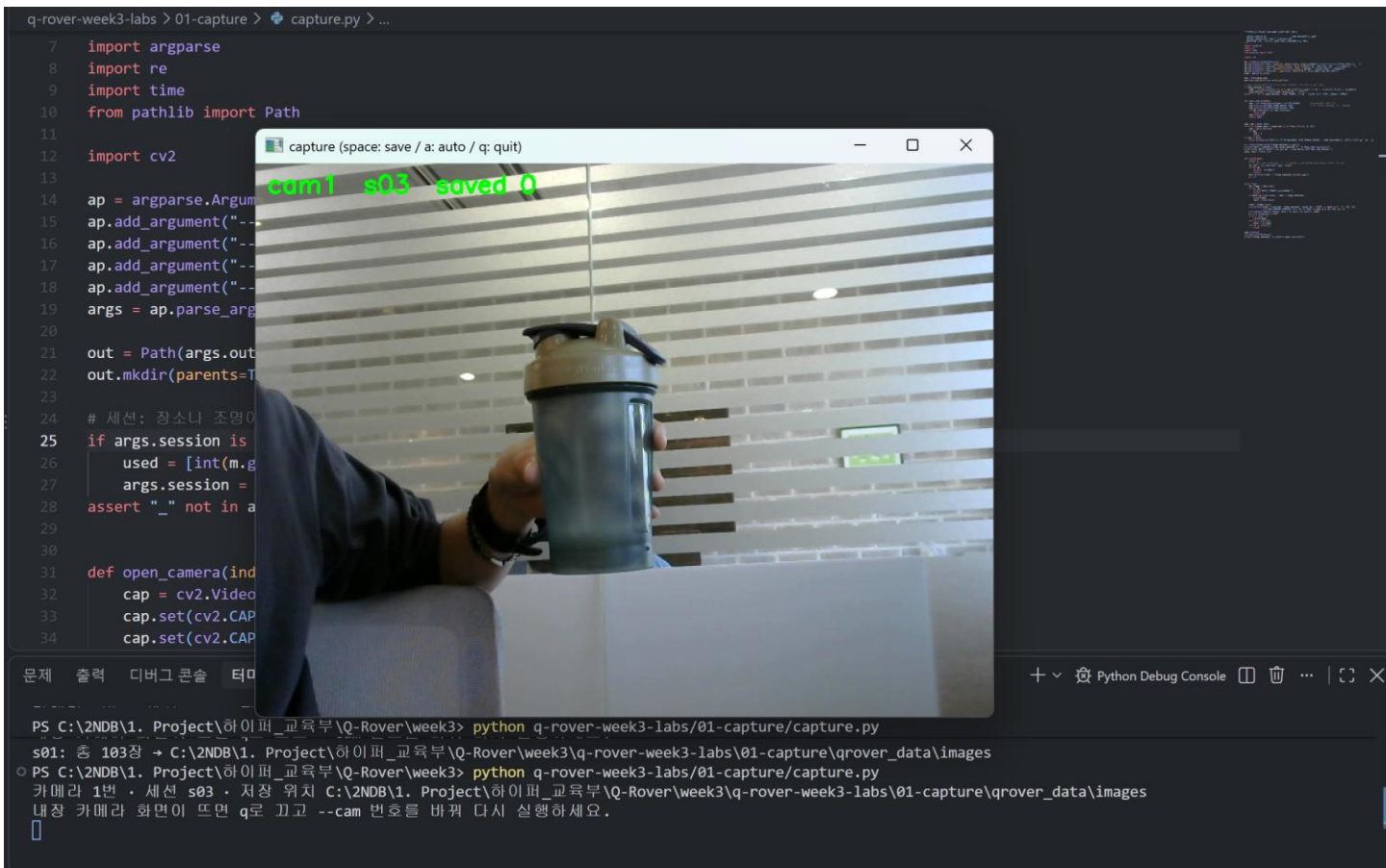
촬영 실행과 키

01-capture 폴더에서
python capture.py 실행

스페이스: 1장 저장
a: 0.5초 자동 촬영
q: 종료

촬영이 정상적으로 진행된다면, 우측 상단
saved 숫자가 올라갑니다.

카메라 번호 자동 선택
1, 0, 2 순서로 시도



동작 확인 리허설 캡처입니다. 학습용 사진은 로버에 장착한 카메라의 높이·각도를 유지합니다.

세션 이름과 촬영량

같은 장소·조명에서 찍은 묶음이 세션

장소나 조명이 바뀌면 프로그램을 종료하고 다시 실행합니다.
파일은 s01_0000.jpg, s02_0000.jpg처럼 세션이 분리되어 저장됩니다.

클래스(물체)마다 사진 100장 이상, 세션 3개 이상 권장

최소 2세션이 필요하며, 그중 한 세션을 통째로 검증에 사용합니다.
물체 없는 배경 사진도 전체의 약 10%를 넣습니다.

촬영 시 주의사항

촬영 시 카메라와 물체 사이의 거리, 물체의 각도, 조명, 조건은 모두 다양하게 변경해가며 촬영합니다.
다양성이 확보되지 않은 데이터로 학습시키면, 로버가 새로운 환경에서 물체를 인식하지 못합니다.

기본 저장: 01-capture/grover_data/images/ 직접 지정: `python capture.py --cam 1 --session s03`

세션별 기록 예시

표는 기록 양식입니다. 실제로 촬영한 장소·조명·장수로 채웁니다.

세션	장소·조명	장수	용도
s01	강의실·형광등	팀 측정	학습 후보
s02	복도·다른 배경	팀 측정	학습 후보
s03	시연 공간·다른 시간	팀 측정	검증 후보

체크포인트 1: 세션별 장수 표. 검증 세션을 학습 중에 섞지 않도록 따로 관리합니다.

STEP 2 labels.txt와 클래스 번호

hyper_mascot

한 줄이 클래스 0

촬영이 끝났으면, labels.txt 파일에 촬영한 물체에 대한 라벨을 타이핑합니다.
선택 과제에서 클래스를 늘리면 줄을 추가합니다.
줄 순서가 클래스 번호가 되므로 처음 정한 순서를 유지합니다.

모든 팀원이 같은 파일에 합의 후 작성하기

팀원 각각이 각자의 물체를 학습시키기로 약속한 상황이라면, 같은 파일에 줄 순서와 라벨 이름을 합의 후 작성하기로 합니다.
각자 라벨을 타이핑하면 철자나 클래스 순서가 달라질 수 있습니다.
학습은 실행돼도, 서로 다른 물체를 같은 번호로 가르치게 됩니다.

makesense.ai 라벨링

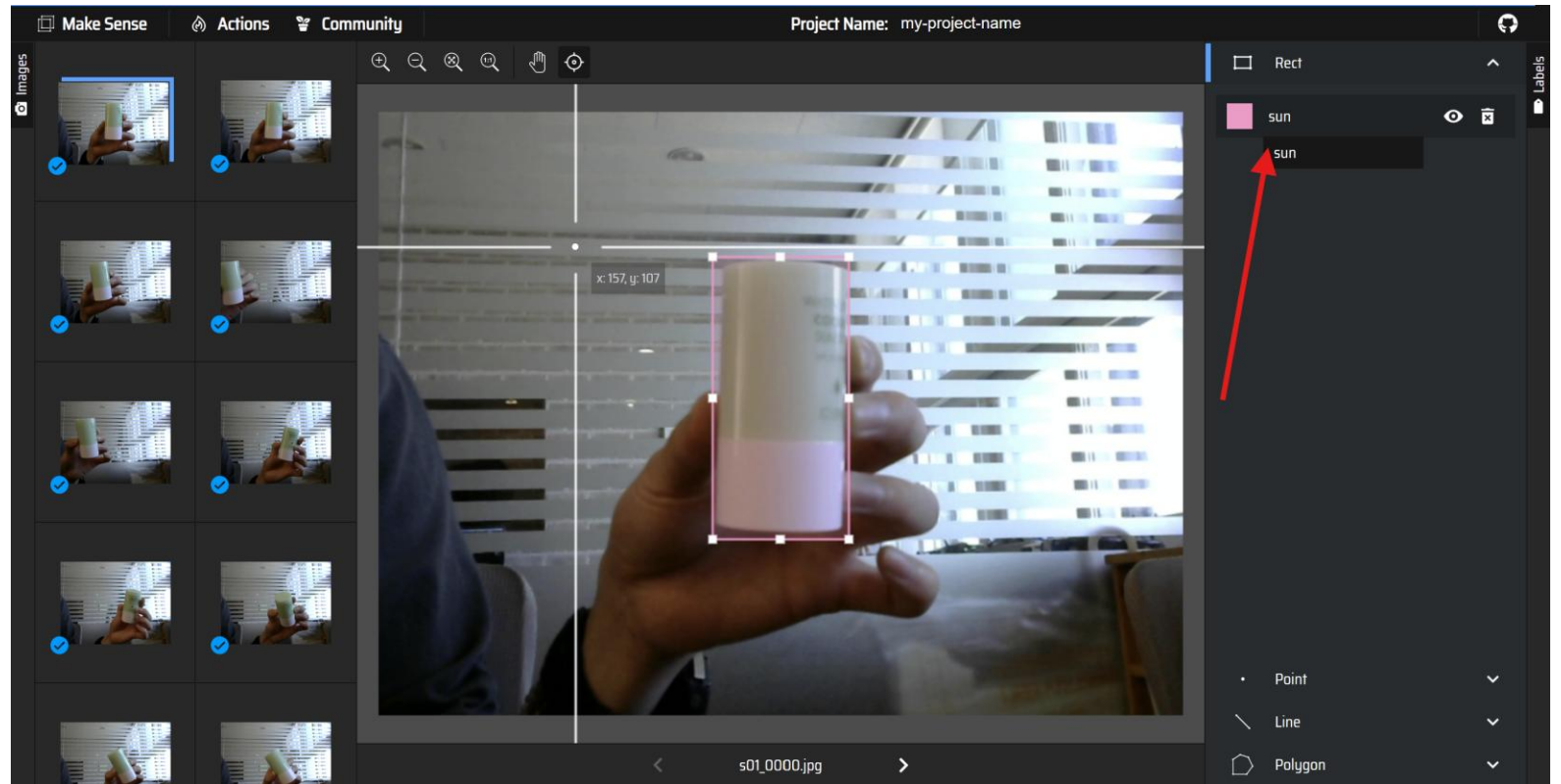
<https://www.makesense.ai/>

1. Get Started

사진 추가

2. Object Detection labels.txt 불러오기

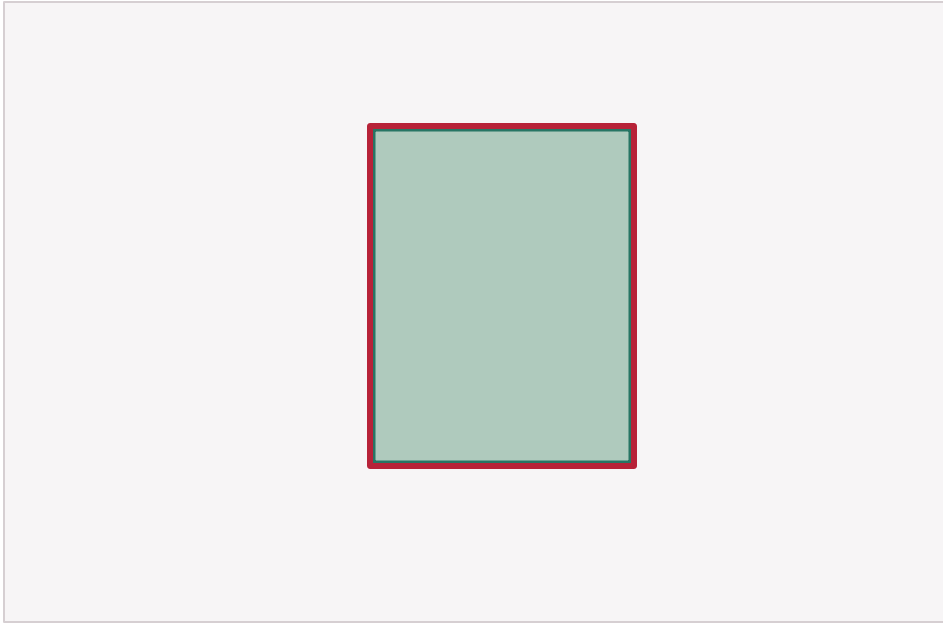
3. 박스마다 라벨 선택 오른쪽 목록 확인



설치 없이 브라우저에서 작업합니다.

* 브라우저에서 직접 라벨링 작업하지 않고도 codex, claude code에 촬영 이미지와 라벨을 알려주고, AI에게 라벨링을 시키는 방법 또한 가능합니다. (정확도가 떨어질 수 있으므로, 이때는 AI가 추출한 결과에서 직접 bounding box 를 수정하며 검수하는 것을 권장합니다.)

라벨링 규칙과 박스



물체에 딱 맞게

모식도: 손·받침대·여백 제외

팀이 합의할 기본 규칙

1. 물체에 딱 맞는 박스
2. 손과 받침대는 제외
3. 절반 이상 보이면 라벨링
4. 박스 높이 15px 미만은 제외
5. 화면 안의 대상은 모두 라벨링

저장·내보내기과 분담

세션 단위로 분담하고 같은 labels.txt 사용

Actions에서 Export Annotations를 선택하고 YOLO 형식(zip)으로 내보냅니다.
처음 약 10장을 작업한 뒤 zip 안에 txt가 있는지 확인합니다.

50장마다 export로 저장

라벨을 선택하지 않은 박스는 export에서 빠집니다.
새로고침·브라우저 종료로 작업이 사라질 수 있습니다.
이어 작업할 때는 같은 사진을 넣고 Import Annotations (YOLO)를 사용합니다.

모든 사진의 박스·라벨을 확인한 뒤 압축 파일을 모읍니다. zip 크기만으로 완료를 판단하지 않습니다.

데이터셋 폴더와 Drive 업로드

```
grover_data/  
  images/  s01_0000.jpg, s01_0001.jpg ...  
  labels/   s01_0000.txt, s01_0001.txt ...  
  labels.txt
```

사진과 라벨 파일의 이름은 같게

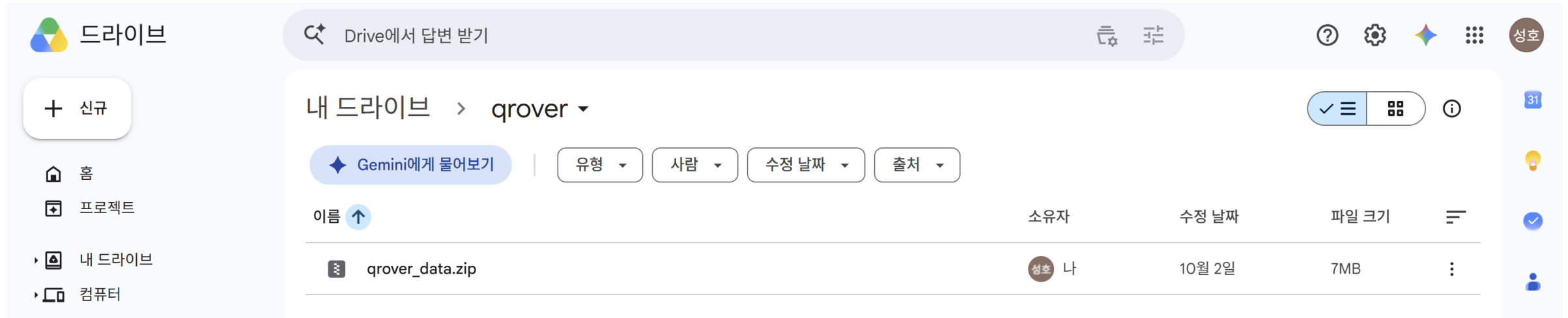
물체 없는 사진의 txt는 없거나 비어 있어도 됩니다.
누락된 라벨이 배경으로 처리되지 않게 확인합니다.

grover_data 폴더째 압축

Google Drive의 내 드라이브/grover/grover_data.zip에 업로드합니다.
images, labels, labels.txt가 압축 파일 안의 grover_data 폴더에 있어야 합니다.

촬영·라벨링이 막히면 01-capture/grover_data.zip의 sun 샘플로 학습 흐름을 먼저 확인할 수 있습니다.

Drive 데이터 준비

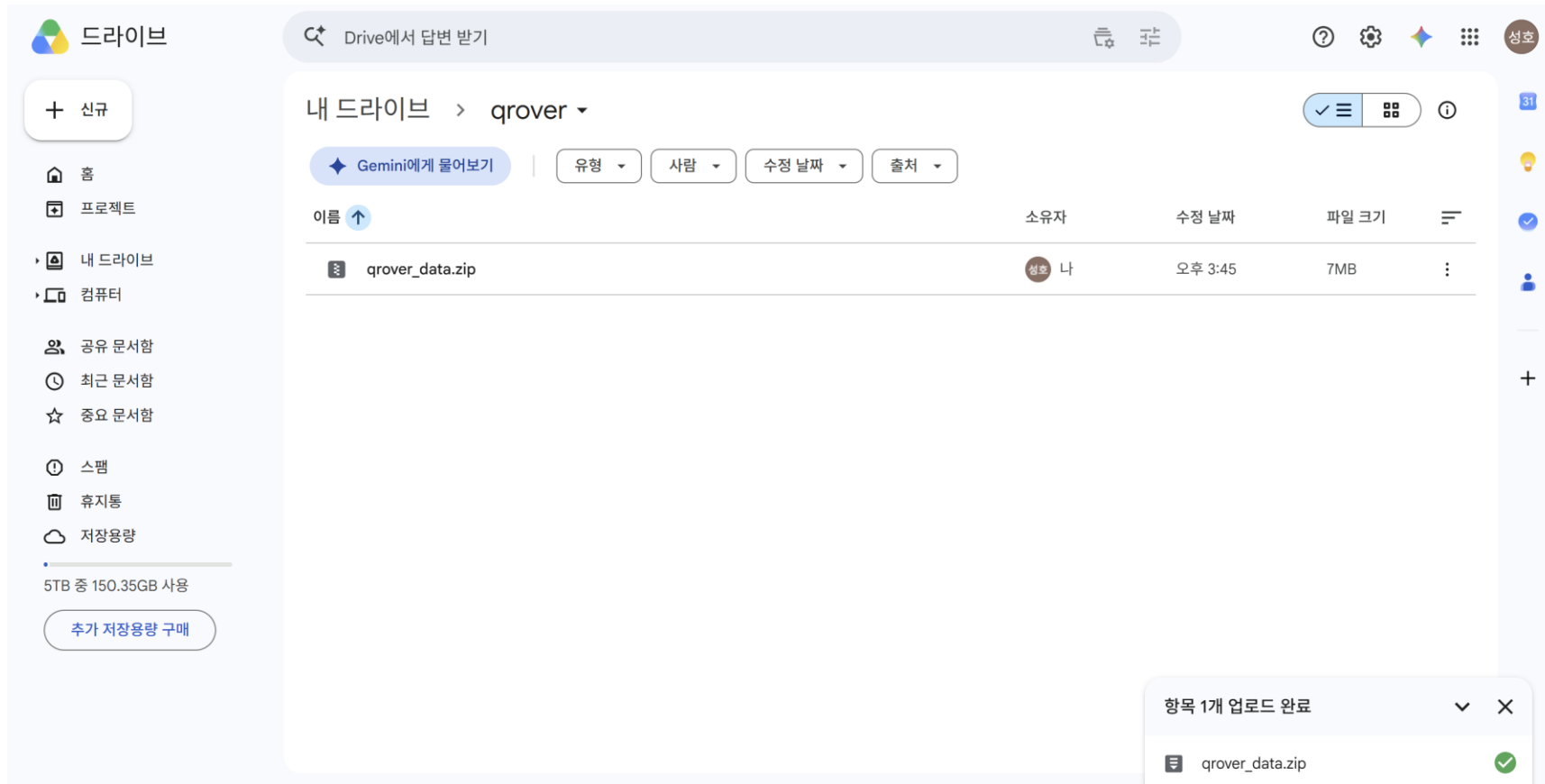


Colab에 로그인한 Google 계정의 내 드라이브 사용

1. 내 드라이브에 grover 폴더를 만듭니다.
2. grover_data 폴더에 압축한 grover_data.zip을 그 안에 올립니다.
3. ZIP 안에는 grover_data/images, grover_data/labels, grover_data/labels.txt가 있어야 합니다.

폴더·파일 이름을 바꾸면 노트북의 MyDrive/grover/grover_data.zip 경로도 같은 이름으로 수정합니다.

Drive 업로드 완료 화면 비교



확인할 것

내 드라이브의
qrover 폴더 안에
qrover_data.zip이
보이는지 확인합니다.

업로드 완료 표시를
확인한 뒤 Colab으로
이동합니다.

파일 크기는 팀이 촬영한 사진 수와 해상도에 따라 달라집니다.

Part 2

학습과 ONNX 확인

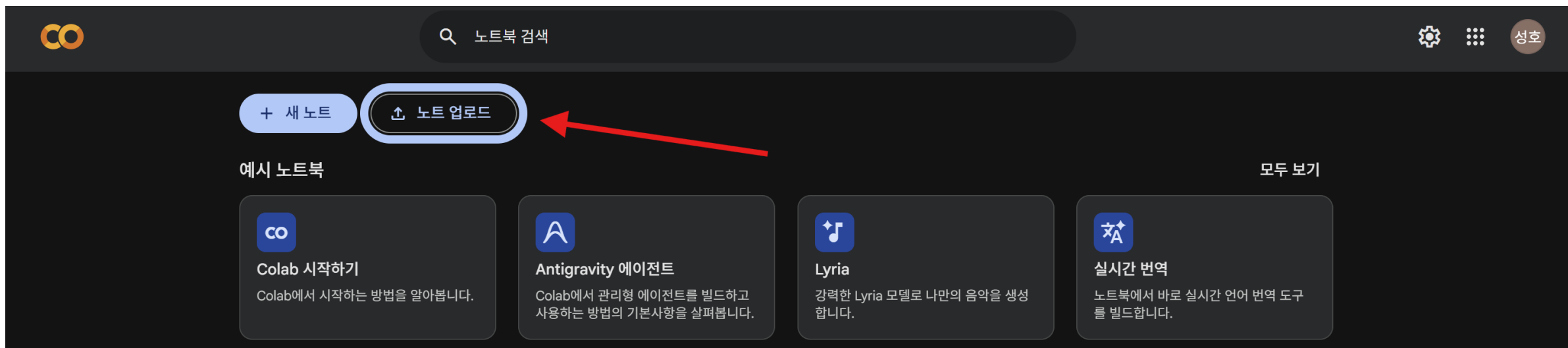
STEP 3 전이학습



이미 배운 시각 특징을 우리 데이터에 맞춥니다.

Colab 접속과 노트북 업로드

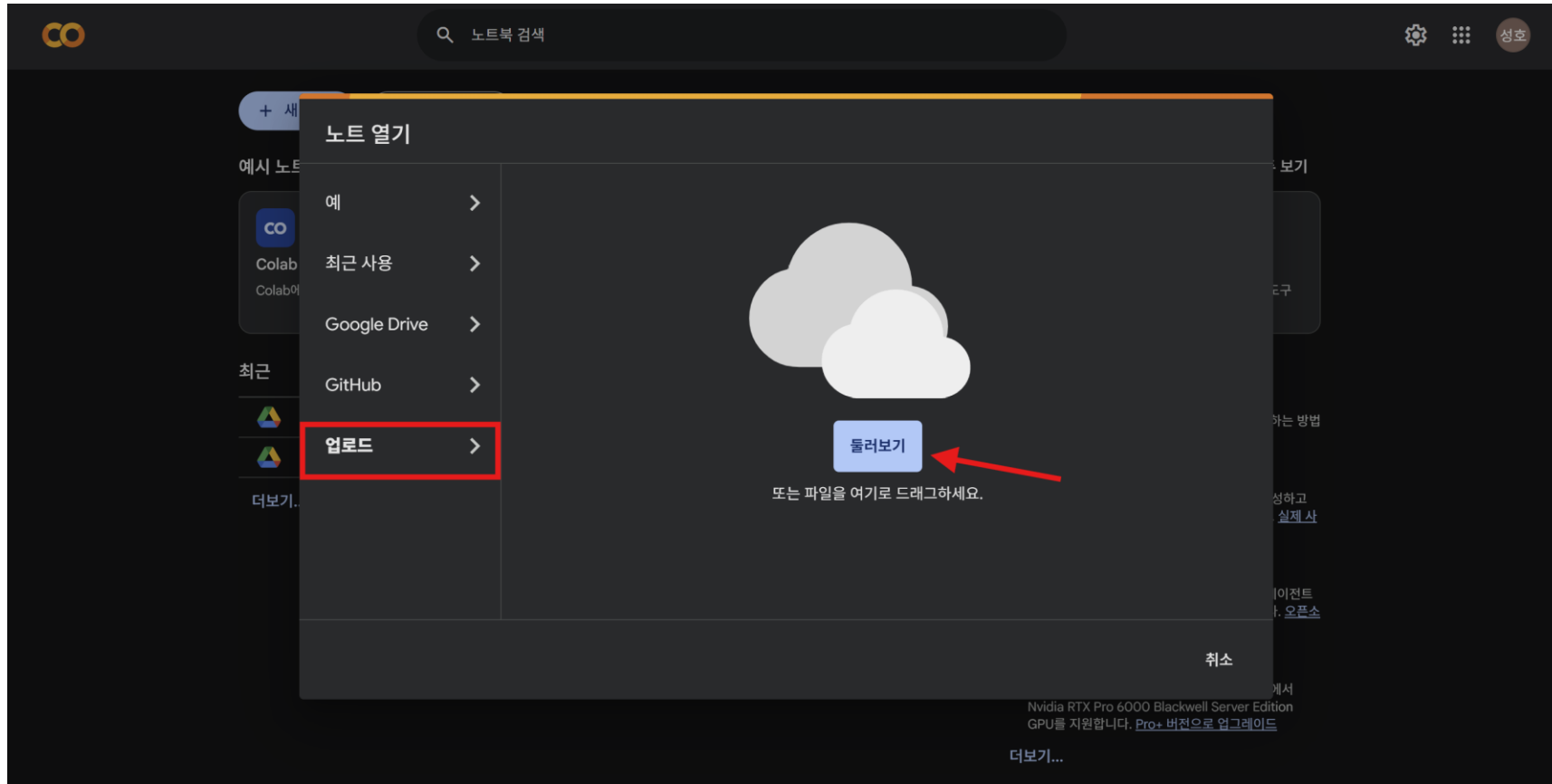
<https://colab.research.google.com/>



Google 계정으로 로그인한 뒤 노트 업로드를 선택합니다.
배포 코드에서 02-train/grover_train.ipynb를 준비합니다.

노트북은 데이터 ZIP과 다른 파일입니다. 먼저 .ipynb를 열고, 첫 셀에서 Drive의 데이터를 가져옵니다.

Colab 업로드 탭에서 파일 열기



확인할 것

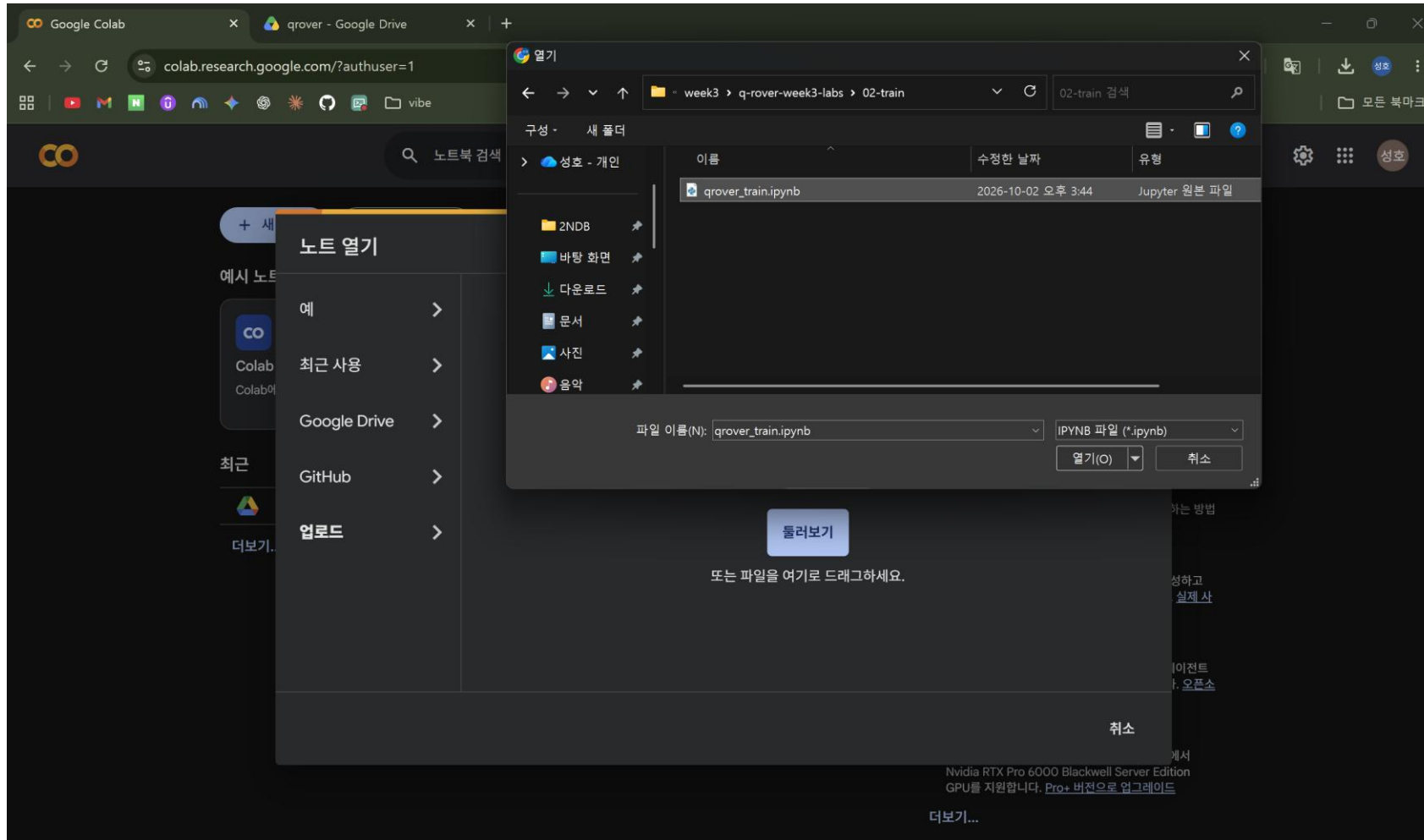
업로드 탭 선택

둘러보기 클릭

02-train 폴더의
grover_train.ipynb
선택

업로드한 노트북을 열면 맨 위 셀부터 마지막 Drive 저장 셀까지 순서대로 실행합니다.

배포 노트북 선택 화면



확인할 것

배포 코드 압축을
룬 폴더에서
02-train 열기

qrover_train.ipynb
선택 후 열기

다음 페이지에서 런타임을 T4 GPU로 바꾼 뒤, 셀을 위에서부터 하나씩 실행합니다.

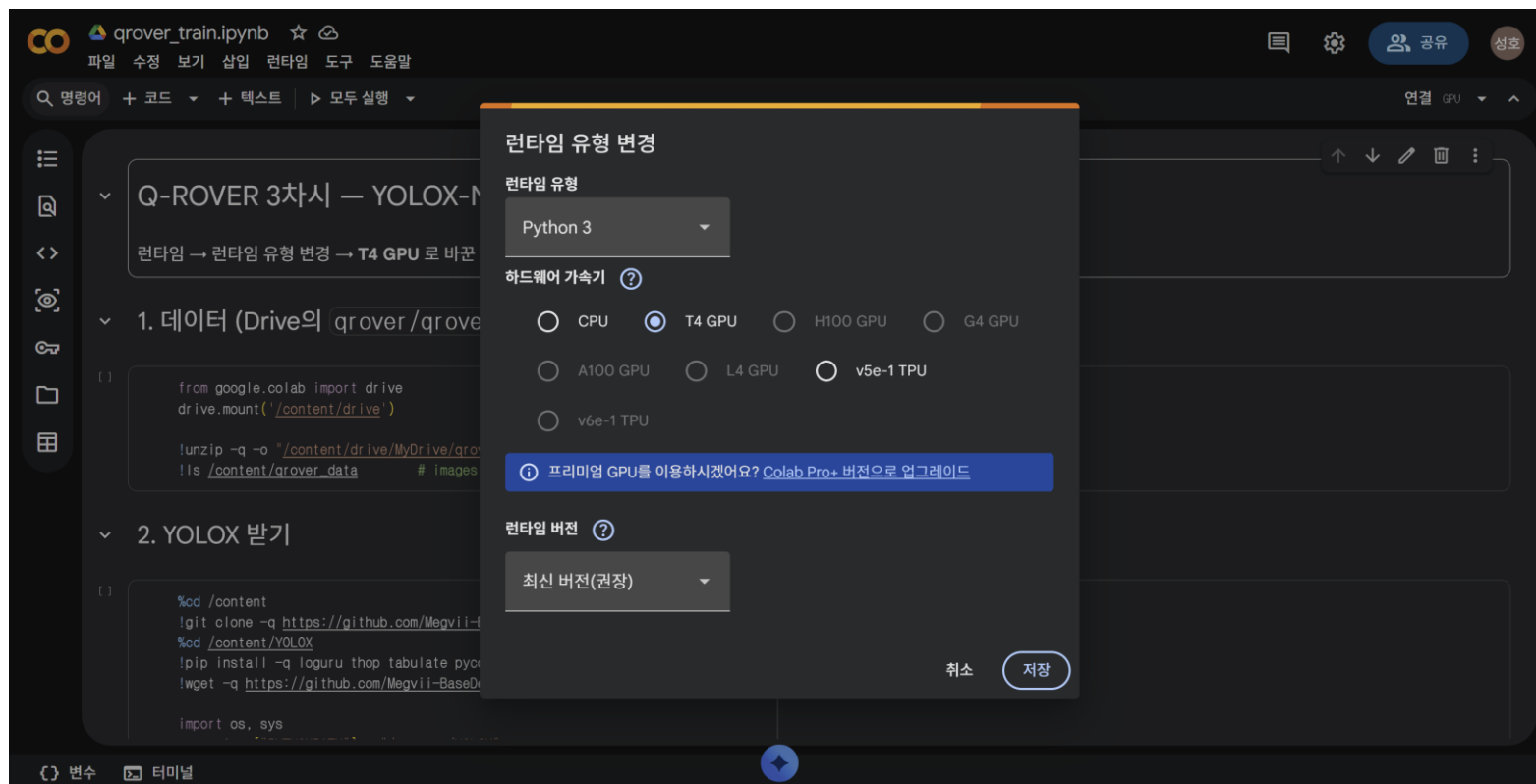
Colab 노트북과 GPU

<https://colab.research.google.com/>

02-train의
grover_train.ipynb를
Colab에서 열기

런타임 유형 변경
T4 GPU 선택

위에서부터 셀 실행
GPU True 확인



무료 GPU 사용 가능 여부와 실행 시간은 달라집니다. 중간에 멈추면 오류 셀과 실행 순서를 확인합니다.

셀 실행과 Drive 연결 확인

각 코드 셀 왼쪽의 실행 버튼을 누르고, 완료되면 다음 셀로 내려갑니다.
Drive 연결 권한은 데이터 ZIP을 올린 Google 계정으로 허용합니다.

1. 데이터 (Drive의 grover/grover_data.zip)

[1]
27초
✓

 `from google.colab import drive
drive.mount('/content/drive')`

`!unzip -q -o "/content/drive/MyDrive/grover/grover_data.zip" -d /content/
!ls /content/grover_data # images labels labels.txt 가 보여야 합니다`

⌵
... Mounted at /content/drive
images labels labels.txt

첫 데이터 셀의 정상 출력

Mounted at /content/drive
images labels labels.txt

Drive 데이터와 YOLOX 준비

Drive 연결 후
grover_data.zip 풀기

images / labels /
labels.txt 확인

YOLOX 0.3.0
GPU True
nano.pth 확인

```
2. YOLOX 받기

%cd /content
git clone -q https://github.com/Megvii-BaseDetection/YOLOX.git
%cd /content/YOLOX
!pip install -q loguru thop tabulate pycocotools onnx onnxruntime
!wget -q https://github.com/Megvii-BaseDetection/YOLOX/releases/download/0.1.1rc0/yolox_nano.pth

import os, sys
os.environ["PYTHONPATH"] = "/content/YOLOX"
sys.path.insert(0, "/content/YOLOX")

/content
/content/YOLOX
61.6/61.6 kB 4.5 MB/s eta 0:00:00
8.9/8.9 MB 78.8 MB/s eta 0:00:00
23.6/23.6 MB 57.7 MB/s eta 0:00:00
343.2/343.2 kB 5.0 MB/s eta 0:00:00

ERROR: pip's dependency resolver does not currently take into account all the packages that are installed. This behaviour is the source of the following dependency conflicts.
ydf 0.15.0 requires protobuf<7.0.0,>=5.29.1, but you have protobuf 7.36.2 which is incompatible.
grpcio-status 1.71.2 requires protobuf<6.0dev,>=5.26.1, but you have protobuf 7.36.2 which is incompatible.
google-ai-generativelanguage 0.6.15 requires protobuf!=4.21.0,!4.21.1,!4.21.2,!4.21.3,!4.21.4,!4.21.5,<6.0.0dev,>=3.20.2, but you have protobuf 7.36.2 which is incompatible.
```

실습 노트북 셀을 그대로 실행합니다. 패키지 경고가 나와도 마지막 확인 출력까지 성공했는지 봅니다.

세션 단위 학습·검증 분리

학습 세션

s01 전체

s02 전체

모델이 배우는 사진

검증 세션

s03 전체

학습에 넣지 않은 조건에서 평가

같은 세션의 비슷한 사진이 양쪽에 들어가면 점수가 부풀 수 있습니다.

약 80:20을 목표로 세션째 나눕니다. 2세션이면 작은 세션 전체를 검증에 사용합니다.

데이터 변환 결과 확인

YOLO txt를 COCO json으로 변환

prepare_dataset.py가 세션 분리, 416×416 squash, 박스 수 집계를 함께 합니다.

```
▼ Overwriting /content/prepare_dataset.py
[0] !python /content/prepare_dataset.py
✓ 1초
▼ [train] 사진 81장 (배경 1장) / 박스 {'sun': 80}
[val] 사진 20장 (배경 0장) / 박스 {'sun': 20}
검증 세션: ['s02'] / 클래스: ['sun']
```

운영진 리허설 예시

학습 81장, 박스 80개
검증 20장, 박스 20개
검증 세션 s02
클래스 sun

팀 결과를 기록합니다. 검증에서 어떤 클래스가 0개라면 VAL_SESSIONS를 다른 세션으로 지정합니다.

실험 설정과 학습 실행

설정	이번 기초 코드	뜻
모델 크기	YOLOX-Nano	depth 0.33 / width 0.25
입력	416×416 / RGB / 0~255	학습·노트북·보드 조건 일치
일정	100 에폭 / 5에폭마다 평가	최고 성능 체크포인트 선택
학습 옵션	-d 1 / -b 16 / --fp16	GPU 1개 / 배치 16 / 절반 정밀도
시작 가중치	-c yolox_nano.pth	COCO 사전학습 가중치



Writing /content/YOLOX/exps/grover_nano.py

Writing 표시가 나오면
exps/grover_nano.py 파일이 만들어진 것입니다.

학습 중 경고와 완료 파일

클래스 예측층 크기 경고는 예상한 동작

COCO의 80개 클래스와 팀의 클래스 수가 달라 마지막 클래스 예측층 크기가 다릅니다. 해당 부분은 새로 배우고, 나머지 맞는 가중치는 사전학습 모델에서 가져옵니다.

YOLOX_outputs/grover_nano/best_ckpt.pth

학습이 끝나면 체크포인트, train_log.txt, 평가 그래프를 확인합니다.

학습 시간은 GPU와 데이터량에 따라 다릅니다. 운영진 리허설은 T4에서 약 6분이었습니다.

학습 명령과 export 셀은 grover_train.ipynb에 들어 있습니다. 실행이 끝나면 Drive 저장 셀까지 진행합니다.

학습 완료 로그 비교

5. 학습 (10~20분)

```
%cd /content/YOLOX
!python tools/train.py -f exps/grower_nano.py -d 1 -b 16 --fp16 -c yolox_nano.pth

Average Precision (AP) @[ IoU=0.50:0.95 | area= large | maxDets=100 ] = 0.770
Average Recall (AR) @[ IoU=0.50:0.95 | area= all | maxDets= 1 ] = 0.765
Average Recall (AR) @[ IoU=0.50:0.95 | area= all | maxDets= 10 ] = 0.765
Average Recall (AR) @[ IoU=0.50:0.95 | area= all | maxDets=100 ] = 0.765
Average Recall (AR) @[ IoU=0.50:0.95 | area= small | maxDets=100 ] = -1.000
Average Recall (AR) @[ IoU=0.50:0.95 | area= medium | maxDets=100 ] = 0.750
Average Recall (AR) @[ IoU=0.50:0.95 | area= large | maxDets=100 ] = 0.825
per class AP:
| class | AP |
|:-----|:-----|
| sum | 71.140 |
per class AR:
| class | AR |
|:-----|:-----|
| sum | 76.500 |

2026-10-02 07:15:33 | INFO | yolox.core.trainer:402 - Save weights to ./YOLOX_outputs/grower_nano
2026-10-02 07:15:34 | INFO | yolox.core.trainer:218 - ==> start train epoch100
2026-10-02 07:15:35 | INFO | yolox.core.trainer:270 - epoch: 100/100, iter: 5/6, gpu mem: 2050Mb, mem: 5.6Gb, iter_time: 0.245s, data_time: 0.002s, total_loss: 1.0, iou_loss: 0.5, tl_loss: 0.1, conf_loss: 0.2, cls_loss: 0.2, lr: 1.250e-04, size: 416, ETA: 0:00:00
2026-10-02 07:15:36 | INFO | yolox.core.trainer:402 - Save weights to ./YOLOX_outputs/grower_nano
100% 2/2 [00:00:00.00, 5.00it/s]
2026-10-02 07:15:36 | INFO | yolox.evaluators.coco_evaluator:259 - Evaluate in main process...
2026-10-02 07:15:36 | INFO | yolox.evaluators.coco_evaluator:292 - Loading and preparing results...
2026-10-02 07:15:36 | INFO | yolox.evaluators.coco_evaluator:292 - DONE (t=0.00s)
2026-10-02 07:15:36 | INFO | pycocotools.coco:367 - creating index...
2026-10-02 07:15:36 | INFO | pycocotools.coco:367 - index created!
2026-10-02 07:15:36 | WARNING | yolox.evaluators.coco_evaluator:298 - Use standard COCOeval.
2026-10-02 07:15:36 | INFO | yolox.evaluators.coco_evaluator:301 - Running per image evaluation...
2026-10-02 07:15:36 | INFO | yolox.evaluators.coco_evaluator:301 - Evaluate annotation type "bbox"
2026-10-02 07:15:36 | INFO | yolox.evaluators.coco_evaluator:301 - DONE (t=0.01s)
2026-10-02 07:15:36 | INFO | yolox.evaluators.coco_evaluator:302 - Accumulating evaluation results...
2026-10-02 07:15:36 | INFO | yolox.evaluators.coco_evaluator:302 - DONE (t=0.01s)
2026-10-02 07:15:36 | INFO | yolox.core.trainer:381 -
Average forward time: 1.71 ms, Average NMS time: 0.83 ms, Average inference time: 2.54 ms
Average Precision (AP) @[ IoU=0.50:0.95 | area= all | maxDets=100 ] = 0.742
Average Precision (AP) @[ IoU=0.50 | area= all | maxDets=100 ] = 1.000
Average Precision (AP) @[ IoU=0.75 | area= all | maxDets=100 ] = 0.939
Average Precision (AP) @[ IoU=0.50:0.95 | area= small | maxDets=100 ] = -1.000
Average Precision (AP) @[ IoU=0.50:0.95 | area= medium | maxDets=100 ] = 0.722
Average Precision (AP) @[ IoU=0.50:0.95 | area= large | maxDets=100 ] = 0.826
Average Recall (AR) @[ IoU=0.50:0.95 | area= all | maxDets= 1 ] = 0.775
Average Recall (AR) @[ IoU=0.50:0.95 | area= all | maxDets= 10 ] = 0.780
Average Recall (AR) @[ IoU=0.50:0.95 | area= all | maxDets=100 ] = 0.780
Average Recall (AR) @[ IoU=0.50:0.95 | area= small | maxDets=100 ] = -1.000
Average Recall (AR) @[ IoU=0.50:0.95 | area= medium | maxDets=100 ] = 0.762
Average Recall (AR) @[ IoU=0.50:0.95 | area= large | maxDets=100 ] = 0.850
per class AP:
| class | AP |
|:-----|:-----|
| sum | 74.162 |
per class AR:
| class | AR |
|:-----|:-----|
| sum | 78.000 |

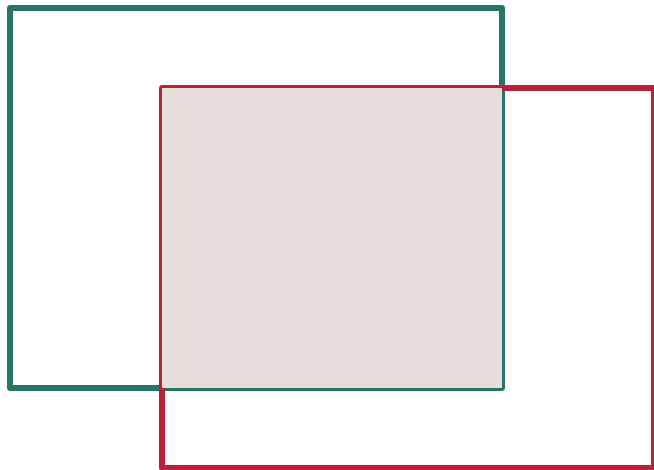
2026-10-02 07:15:36 | INFO | yolox.core.trainer:402 - Save weights to ./YOLOX_outputs/grower_nano
2026-10-02 07:15:36 | INFO | yolox.core.trainer:200 - Training of experiment is done and the best AP is 75.67
```

100/100 에폭과 학습 완료 문구를 확인합니다. 사진의 best AP 75.67은 예시이며 팀 결과를 기록합니다.

AP50과 IoU의 의미

IoU = 두 박스의 교집합 면적 / 합집합 면적

정답 박스



예측 박스

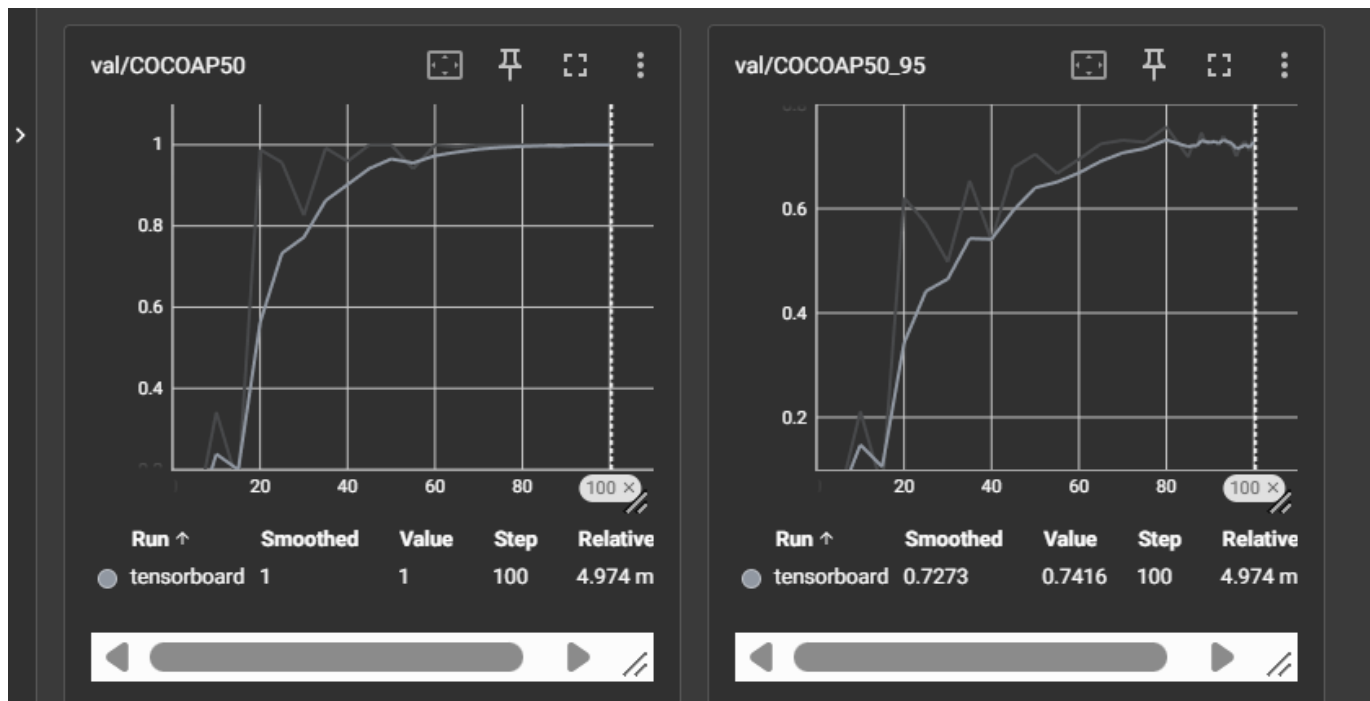
AP50

IoU 0.50 이상인 예측을 정답으로 인정하는 기준입니다.
여러 신뢰도 임계값에서 precision과 recall을 계산하고
그 곡선의 면적으로 AP를 구합니다.

클래스 평균 AP50을 전체 성능 지표로 봅니다.
단순히 맞힌 사진 비율과는 계산이 다릅니다.

높은 AP50과 실제 주행 성공은 별도로 확인합니다. 작은 검증 세트는 새 환경의 성능을 충분히 보여주지 못합니다.

실제 리허설 학습 결과



```
!grep "IoU=0.50 " YOLOX_outputs/grover_nano/train_log.txt | tail -1  
!grep -A 6 "per class AP" YOLOX_outputs/grover_nano/train_log.txt | tail -7
```

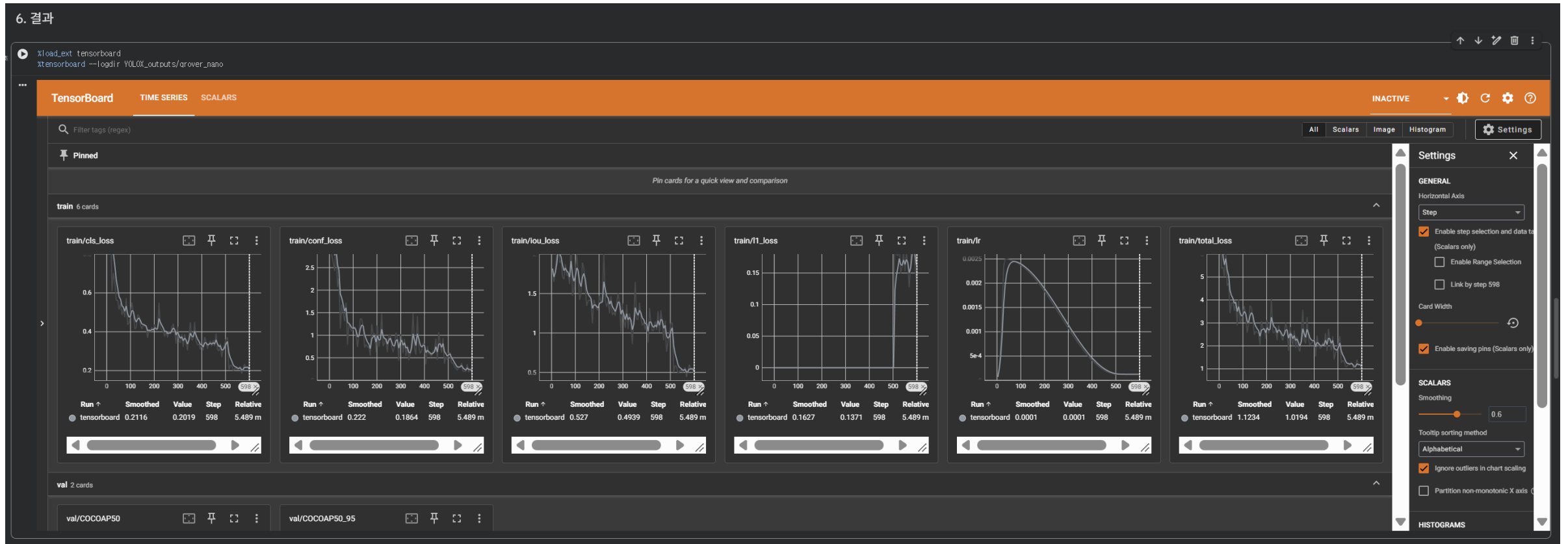
... Average Precision (AP) @[IoU=0.50 | area= all | maxDets=100] = 1.000
per class AP:
class	AP
sun	74.162
per class AR:	
class	AR
:-----	:-----

AP50 1.000, 마지막 평가 AP50:95 0.742

예시: 1클래스 sun / 학습 81장 / 검증 20장

운영진 리허설 기록입니다. 우리 팀 수치와 구분하고, STEP 4.5에서 실제 검출을 확인합니다.

TensorBoard 학습 곡선 비교



train/total_loss와 각 손실의 흐름을 확인하고, 검증 AP 그래프도 함께 기록합니다.

전처리는 세 환경이 같아야 함

크기·색·값 범위가 달라도 프로그램은 실행될 수 있어 성능만 나빠지기도 합니다.

조건	Colab 학습	노트북 확인	Edge Impulse
크기	416×416 squash	cv2.resize squash	Resize mode: Squash
색 순서	RGB 데이터셋	BGR를 RGB로 변환	RGB
값 범위	0~255 그대로	float32 / 0~255	Pixels ranging 0..255

RGB 변환을 빼면 Colab은 BGR로 배우고 보드는 RGB를 입력합니다. 빨강·파랑 채널을 서로 다르게 보게 됩니다.

ONNX 내보내기과 저장



입력: (1, 3, 416, 416)

출력: (1, 3549, 5 + 클래스 수)

1클래스는 마지막 크기가 6, 3클래스는 8입니다.

export는 decode=False로 칸 좌표를 유지합니다.

Drive grover/out에 .onnx, best_ckpt.pth, train_log.txt를 저장합니다. /content는 세션 종료 시 사라질 수 있습니다.

ONNX 내보내기 실행 결과

7. ONNX 내보내기

```
%%writefile /content/export_grover.py
'''합승된 체크포인트(.pth) → ONNX
YOLOX의 tools/export_onnx.py는 최신 PyTorch에서 동작하지 않아(torch.onnx._export 삭제) 같은 일을 하는 짧은 버전을 씁니다.

python export_grover.py exps/grover_nano.py YOLOX_outputs/grover_nano/best_ckpt.pth grover_nano.onnx
'''

import sys

import torch
from torch import nn

from yolox.exp import get_exp
from yolox.models.network_blocks import SiLU
from yolox.utils import replace_module

exp_file, ckpt_file, out_file = sys.argv[1:4]
exp = get_exp(exp_file, None)
model = exp.get_model()

ckpt = torch.load(ckpt_file, map_location="cpu", weights_only=False) # 직접 학습한 파일이라 믿고 엽니다
model.load_state_dict(ckpt["model"] if "model" in ckpt else ckpt)
model.eval()
model = replace_module(model, nn.SiLU, SiLU) # SiLU를 x=sigmoid(x)로 풀어서 내보냄
model.head.decode_in_inference = False # 칸(grid) 좌표 그대로 내보냄 - Edge Impulse의 YOLOX 형식이 이것

dummy = torch.randn(1, 3, *exp.test_size)
kwargs = dict(input_names=["images"], output_names=["output"], opset_version=11)
try:
    torch.onnx.export(model, dummy, out_file, dynamo=False, **kwargs) # PyTorch 2.5 이상
except TypeError:
    torch.onnx.export(model, dummy, out_file, **kwargs) # 그보다 오래된 PyTorch

with torch.no_grad():
    shape = tuple(model(dummy).shape)
    print(f"저장: {out_file} 입력 (1, 3, {exp.test_size[0]}, {exp.test_size[1]}) 출력 {shape}"
          f" = (1, 칸 개수, 4+클래스 {exp.num_classes}개)")

Writing /content/export_grover.py
```

python /content/export_grover.py exps/grover_nano.py YOLOX_outputs/grover_nano/best_ckpt.pth /content/grover_nano.onnx

... /content/export_grover.py:28: DeprecationWarning: You are using the legacy TorchScript-based ONNX export. Starting in PyTorch 2.9, the new torch.export-based ONNX exporter has become the default. Learn more about the new export logic: https://docs.pytorch.org/docs/stable/onnx_export.html. For exporting control flow: https://pytorch.org/tutorials/beginner/onnx/export_control_flow.html.
 torch.onnx.export(model, dummy, out_file, dynamo=False, **kwargs) # PyTorch 2.5 이상
저장: /content/grover_nano.onnx 입력 (1, 3, 416, 416) 출력 (1, 3549, 5) = (1, 칸 개수, 4+클래스 1개)

생성 파일 grover_nano.onnx와 출력 크기 (1, 3549, 5 + 클래스 수)를 확인합니다.

사진은 sun 1클래스 예시로 마지막 크기가 6입니다. 클래스 수가 바뀌면 마지막 크기도 달라집니다.

검증 셀 오류 예시와 Drive 저장

```
✓ 0초 Writing /content/onnx_check.py

[14]
✓ 0초 !python /content/onnx_check.py /content/qrover_nano.onnx /content/qrover_data/labels.txt Wn --images datasets/qrover/val2017 --save /content/check

from IPython.display import Image, display
import glob
for p in sorted(glob.glob("/content/check/*.jpg"))[:8]:
    display(Image(p))

usage: onnx_check.py [-h] [--cam CAM] [--images IMAGES] [--save SAVE]
                    [--conf CONF]
                    model labels
onnx_check.py: error: unrecognized arguments: n

9. Drive에 저장 (꼭 실행)

[15]
✓ 0초 !mkdir -p /content/drive/MyDrive/qrover/out
!cp /content/qrover_nano.onnx YOLOX_outputs/qrover_nano/best_ckpt.pth YOLOX_outputs/qrover_nano/train_log.txt /content/drive/MyDrive/qrover/out/
```

unrecognized arguments: n이 나오면 명령에 들어간 문자 \n을 삭제합니다.

```
!python /content/onnx_check.py /content/qrover_nano.onnx /content/qrover_data/labels.txt \
--images datasets/qrover/val2017 --save /content/check
```

검증 셀의 실행 결과를 확인하고 마지막 Drive 저장 셀도 실행합니다. 저장 위치: 내 드라이브/qrover/out

체크포인트/ 기록해야 할 것

학습 지표와 그래프

전체 AP50, 클래스별 AP 표, val/COCOAP50 그래프를 기록합니다.

검증 세션 사진 결과 3장

노트북의 검증 셀로 박스 위치와 라벨을 확인합니다.

전체 AP50이 낮으면 라벨 순서·데이터량을, 한 클래스만 낮으면 해당 데이터를 점검합니다.

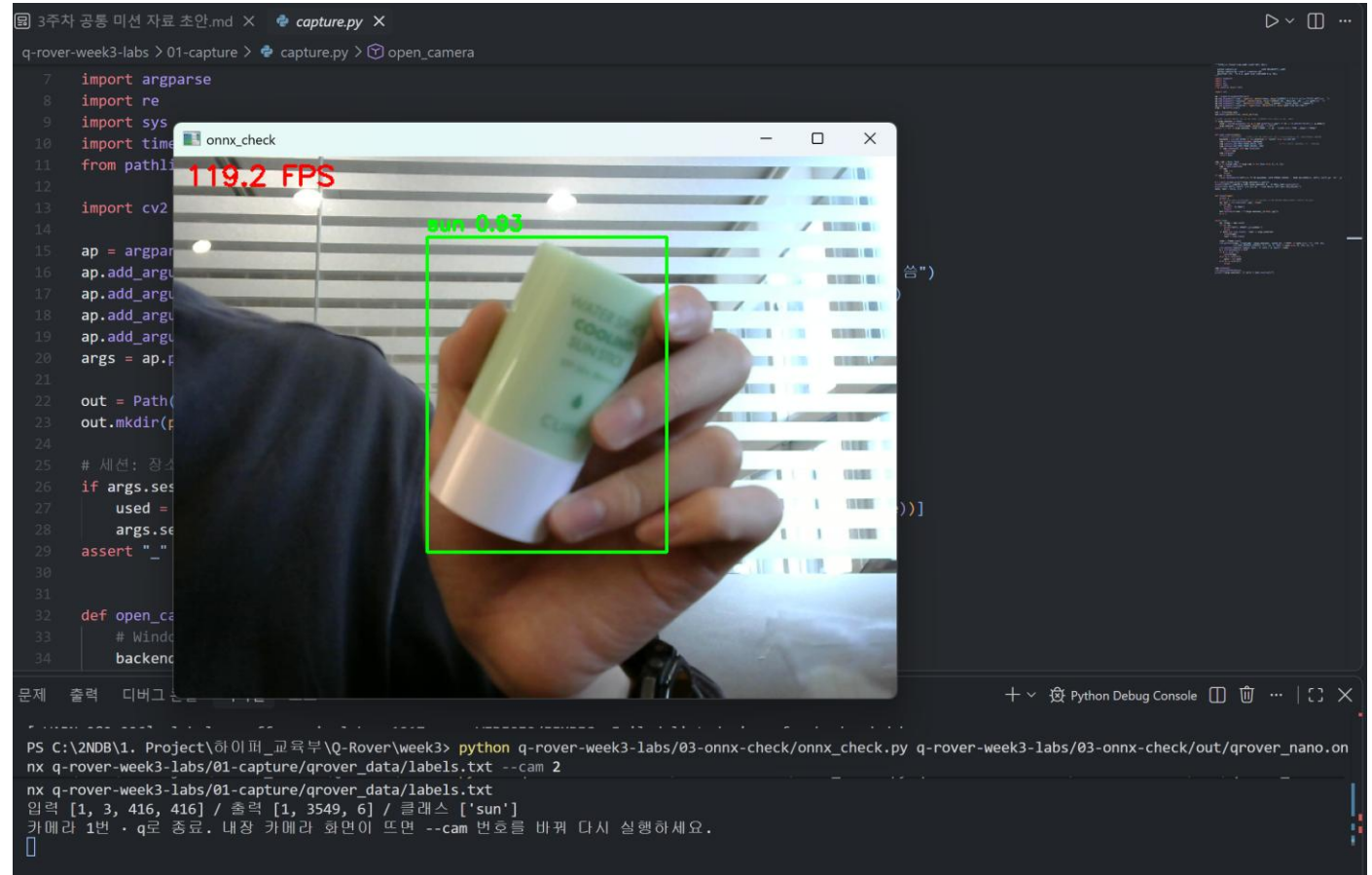
Drive 저장 링크를 README에 남깁니다. 작은 검증 세트에서 얻은 높은 점수도 새 장소·조명에서 다시 확인합니다.

STEP 4 같은 ONNX를 노트북에서

03-onnx-check에서 실행
python onnx_check.py
grover_nano.onnx labels.txt

화면 양끝·가운데와
먼 거리·가까운 거리 확인

왼쪽 위 FPS 기록
q로 종료

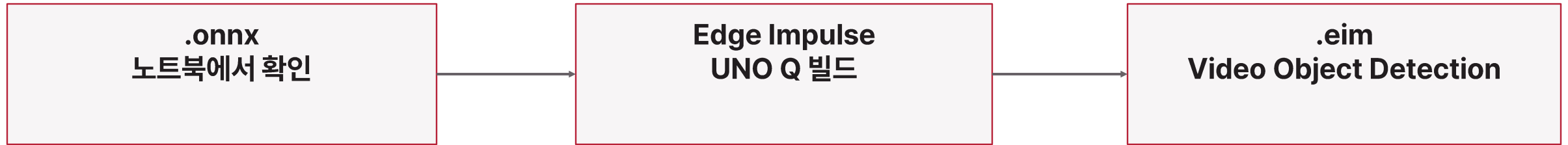


리허설 캡처입니다. 표시 FPS는 검출 계산 시간 기준이며 카메라 촬영·표시 시간을 포함한 전체 FPS와 다릅니다.

Part 3

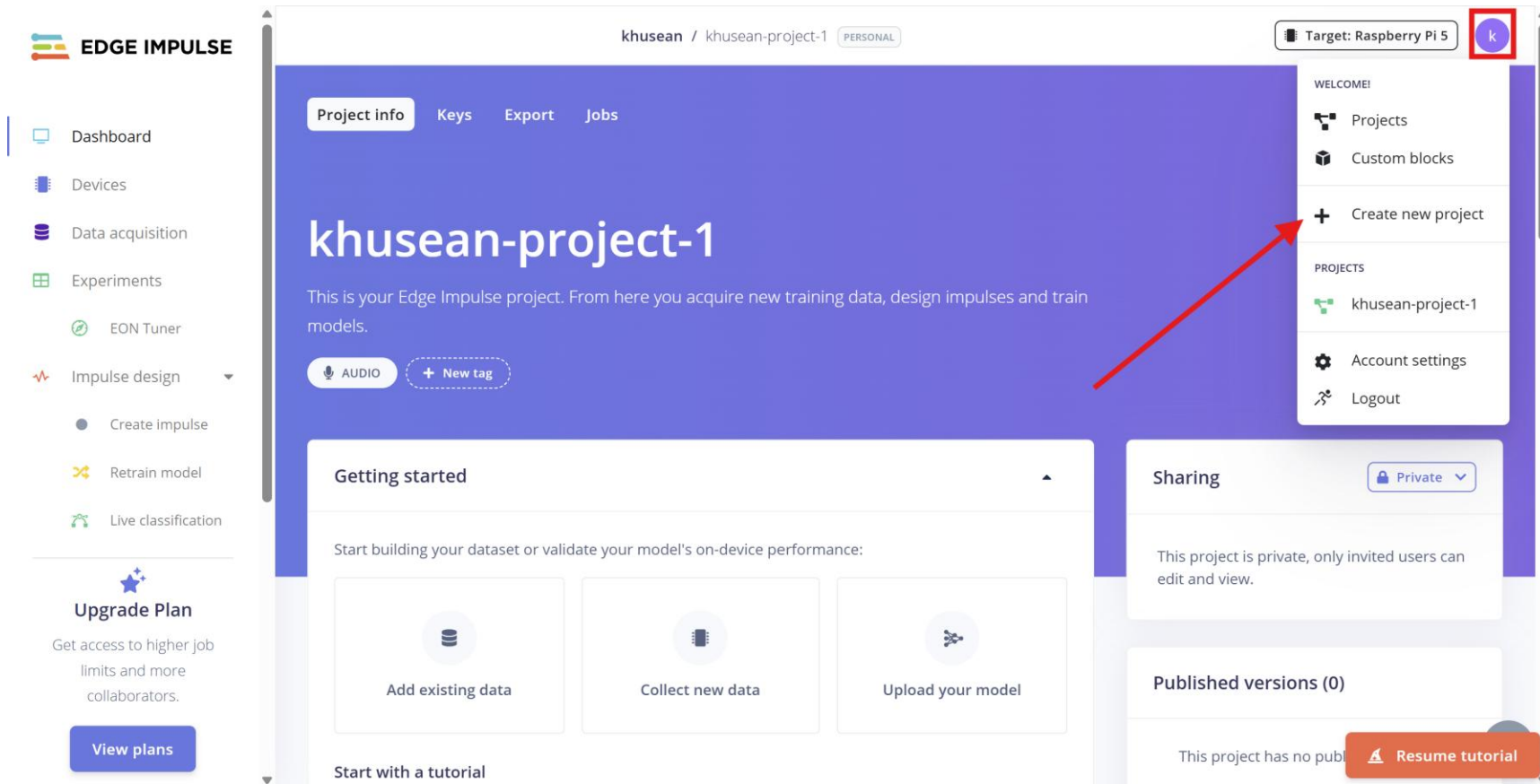
우리 모델을 UNO Q에 배포

STEP 5 모델 형식과 실행 환경



이미 학습한 모델을 보드의 실행 형식으로 변환하는 과정

Edge Impulse 새 프로젝트 메뉴



확인할 것

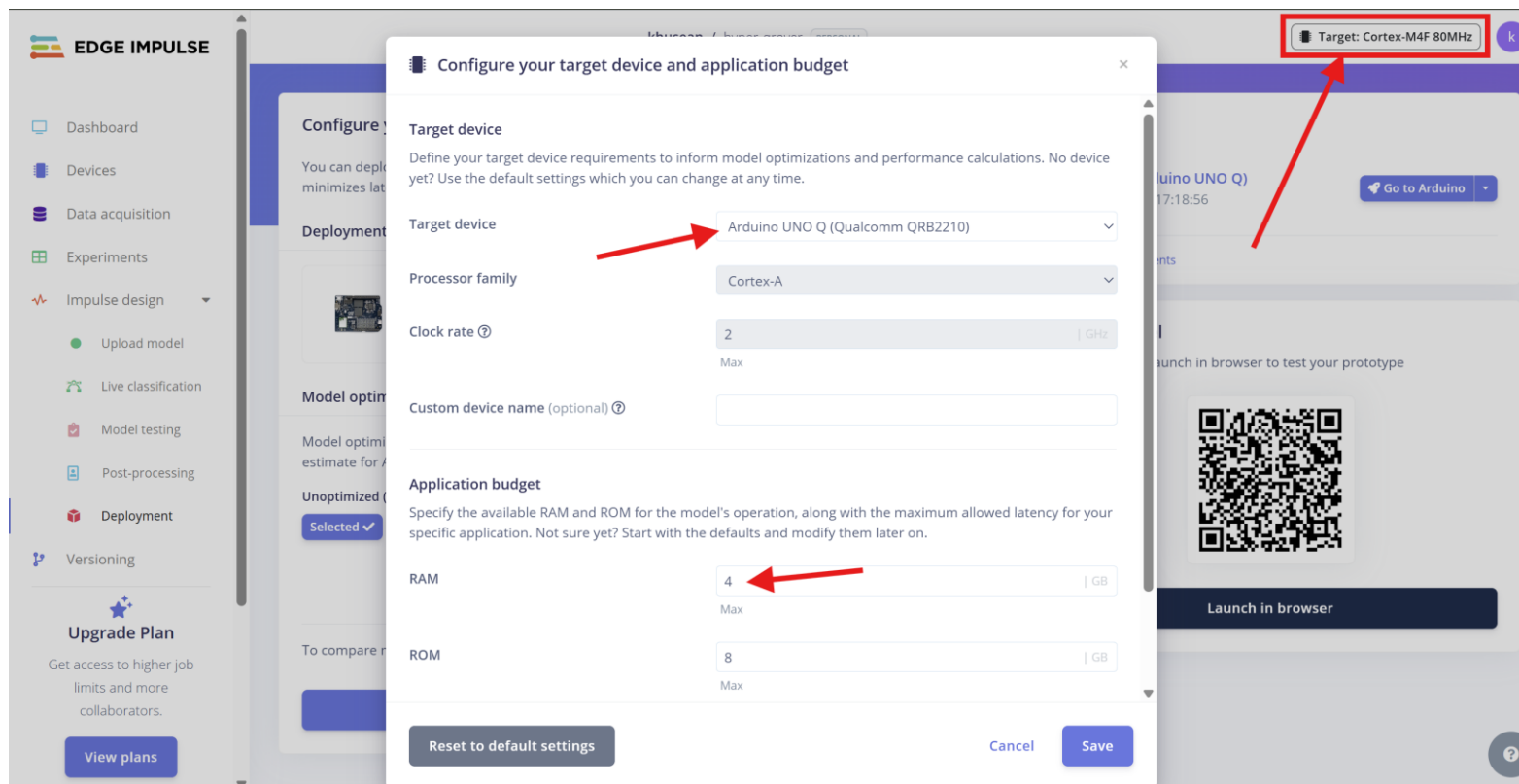
프로필 메뉴에서
Create new project
선택

새 프로젝트의
OBJECT DETECTION
카테고리 확인

사진은 새 프로젝트 메뉴 위치 예시입니다. 팀 프로젝트의 Target은 Arduino UNO Q로 지정합니다.

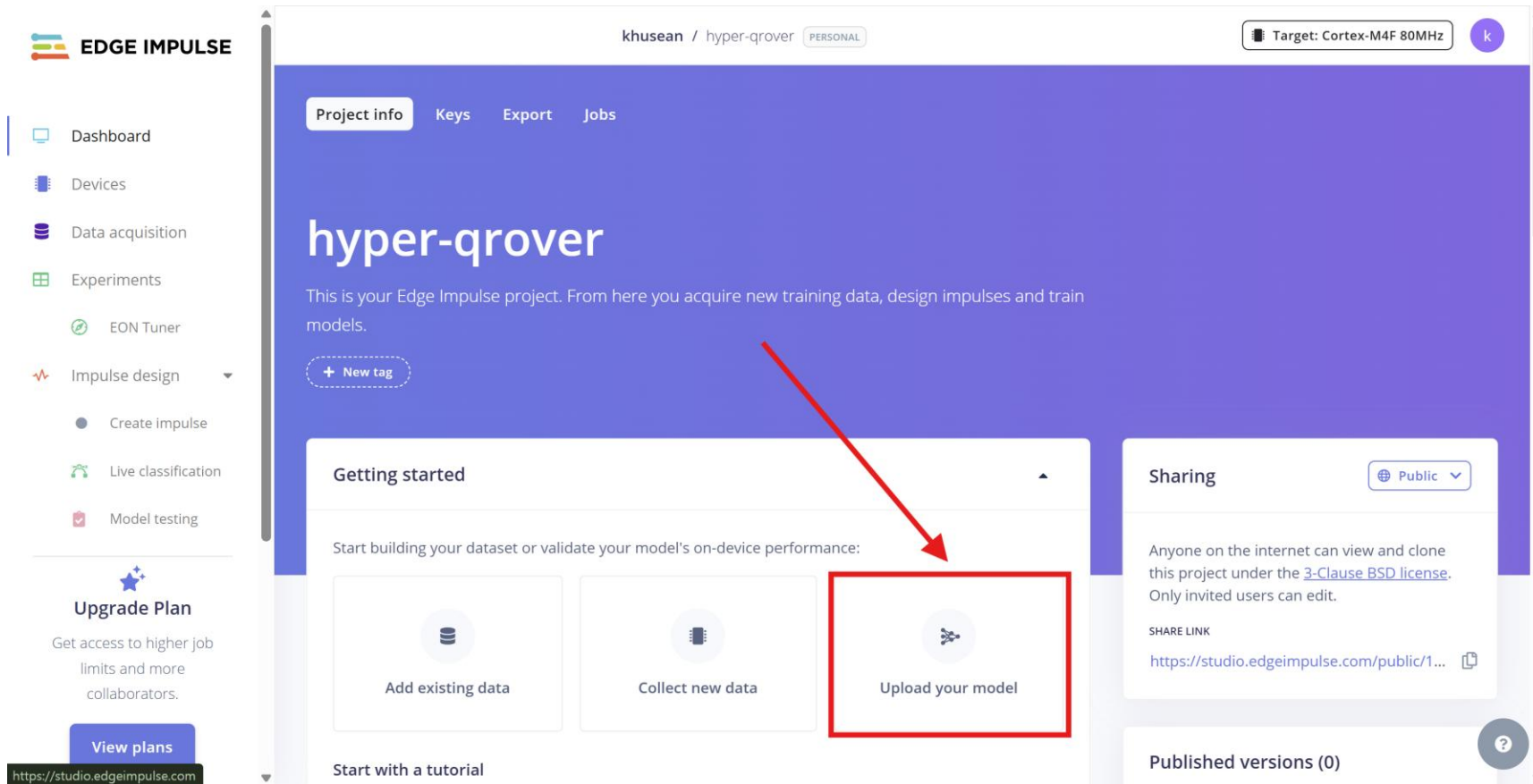
프로젝트 카테고리 Target

1. 브라우저에서 EDGE IMPULSE에 접속
2. 접속 후 로그인(Arduino 계정과 같은 계정 사용/app lab 계정)
3. 새 프로젝트 생성 후 OBJECT DETECTION 확인
4. 화면 상 오른쪽 위 Target에서 Arduino UNO Q선택



Target이나 카테고리가 다르면 App Lab의 Video Object Detection 모델 목록에 보이지 않을 수 있습니다.

Upload your model 메뉴



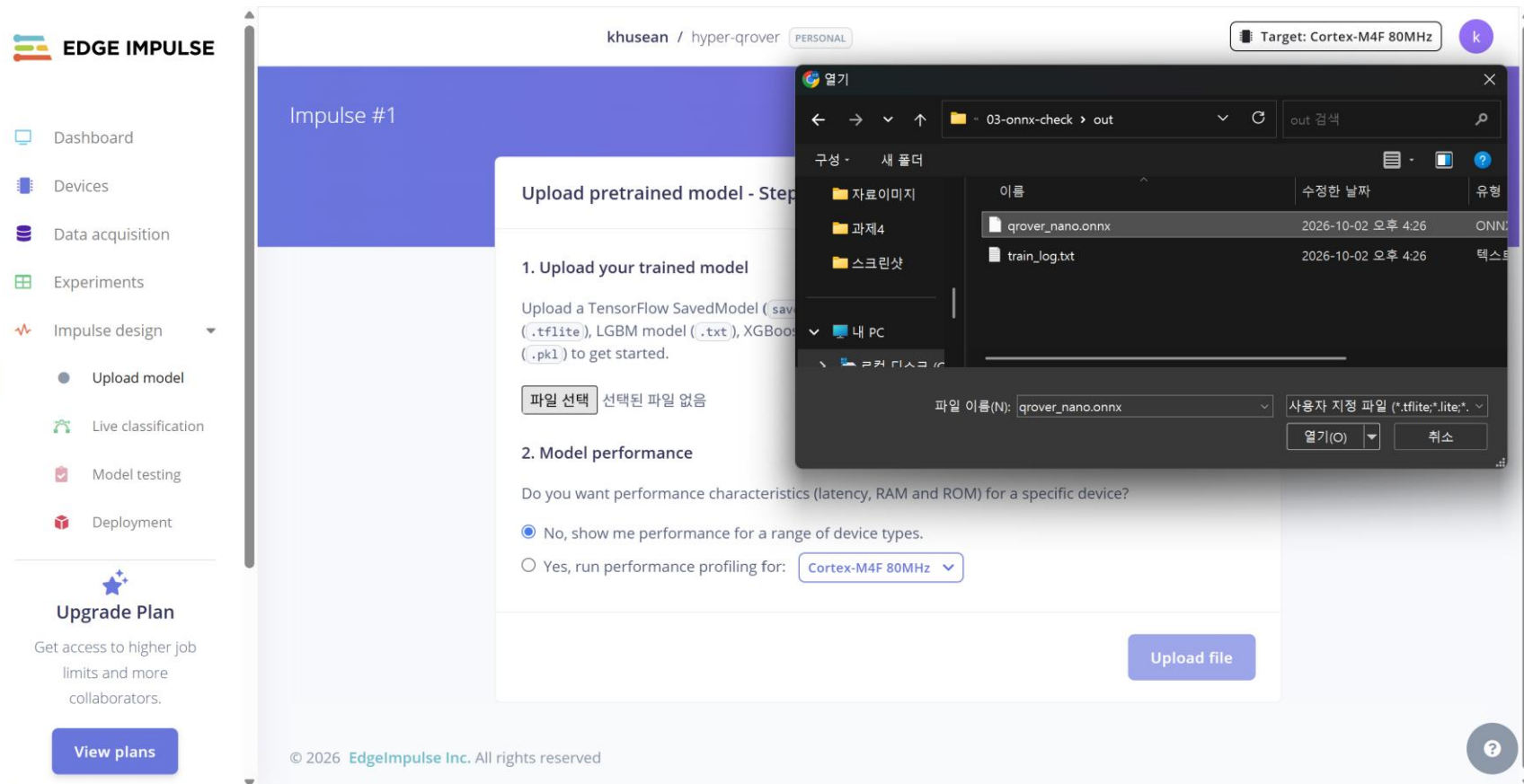
확인할 것

대시보드의
Upload your model
선택

이미 학습한
grover_nano.onnx를
가져옵니다.

사진의 메뉴 위치를 참고합니다. 팀 프로젝트는 Target을 Arduino UNO Q로 바꿉니다.

학습한 ONNX 파일 선택



확인할 것

파일 선택에서
grover_nano.onnx
선택

Drive grover/out의
모델을 노트북에
다운로드해 사용

이어서 Model performance는 기본값을 유지하고, representative features는 비워 둡니다.

학습 모델 업로드

5. Upload your model
grover_nano.onnx 선택

6. Model performance는
기본 옵션 유지

7. Representative features
(Optional)는 비움

-> Upload file 완료

Upload pretrained model - Step 1: Upload a model

1. Upload your trained model

Upload a TensorFlow SavedModel (`saved_model.zip`), ONNX model (`.onnx`), TensorFlow Lite model (`.tflite`), LGBM model (`.txt`), XGBoost model (`.json`), NGBoost model (`ngboost.pkl`) or pickle model (`.pkl`) to get started.

파일 선택

grover_nano.onnx

2. Model performance

Do you want performance characteristics (latency, RAM and ROM) for a specific device?

☒ No, show me performance for a range of device types.

☐ Yes, run performance profiling for:

Cortex-M4F 80MHz ▾

3. Upload representative features *(Optional)*

If you upload a set of representative features - for example, your validation set - as an `.npy` file we can automatically quantize this model for better on-device performance.

파일 선택

선택된 파일 없음

Upload file

이미 노트북에서 검증한 float32 모델을 가져옵니다. 임의의 .npy 파일로 양자화를 추가하지 않습니다.

Edge Impulse 모델 설정

항목	설정값
Model input	Image / (416, 416, 3)
Input scaling	Pixels ranging 0..255 (not normalized)
Resize mode	Squash
Model output / Output layer	Object detection / YOLOX
Score threshold	0.2 (후속 웹 임계값의 하한이 될 수 있음)
Output labels	labels.txt 불러오기, 순서 유지

일반 MCU 성능 경고와 추정 지연 대신 UNO Q의 실제 동작을 측정합니다. 설정은 학습·노트북 전처리와 맞춥니다.

Check model behavior

검증 세션 사진 1장
Upload an image

Test sample 실행
박스.라벨 확인

STEP 4의 같은 사진
결과와 비교

Impulse #1

< Step 2: Process "grover_nano.onnx"

Configure model settings for optimal processing.

Model input
Input shape: (416, 416, 3)

How is your input scaled?
Pixels ranging 0..255 (not normalized)

Resize mode
Squash

Model output
Output shape: (3549, 6)

Output layer
YOLOX

Score threshold
0.2

Output labels (1)
sun

파일 선택 선택된 파일 없음

Save model

On-device performance

MCUs

DEVICE	LATENCY	RAM	ROM	RAM	ROM
Low-end MCU	⚠ This model won't run on MCUs. Calculated arena size is 14MB				
High-end MCU	⚠ This model won't run on MCUs. Calculated arena size is 14MB				
+ AI accelerator	⚠ This model won't run on MCUs. Calculated arena size is 14MB				

Microprocessors

DEVICE	LATENCY	MODEL SIZE
CPU	711 ms.	3.5M
GPU or accelerator	119 ms.	3.5M

Check model behavior

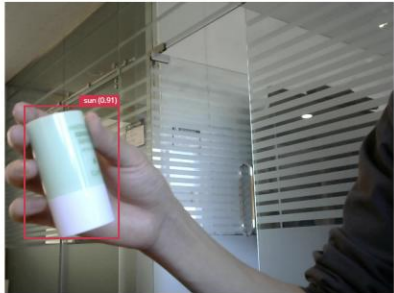
Upload test data to ensure correct model settings and proper model processing. (Optional)

Upload an image
Upload an image to try out your model. The image will be automatically resized to 416x416 (RGB).

파일 선택 s02_0096.jpg

Test sample

Results



Modify model settings if results do not reflect expected model behavior, otherwise save and deploy your model

라벨만 뒤바뀌면 labels.txt 순서, 박스 크기·위치가 이상하면 YOLOX 출력층·입력 스케일을 확인합니다.

Deployment에서 UNO Q Build

Deployment에서
UNO Q 검색.선택

Build 완료 후
Go to Arduino

C++ library zip과
UNO Q 빌드 구분

Configure your deployment

You can deploy your impulse to any device. This makes the model run without an internet connection, minimizes latency, and runs with minimal power consumption. [Read more.](#)

Deployment target

**Arduino UNO Q**
An EIM binary for the Arduino UNO Q CPU that implements the Edge Impulse Linux protocol.

Model optimizations and performance

Model optimizations can increase on-device performance but may reduce accuracy. Performance estimate for Arduino UNO Q (Qualcomm QRB2210).

Unoptimized (float32)

Selected ✓

	TOTAL
LATENCY	154 ms.
RAM	-
FLASH	3.5M
ACCURACY	-

To compare model accuracy, run model testing.

Run model testing

Build

Latest build

**v3 (Arduino UNO Q)**
Today, 17:18:56

Go to Arduino

View all deployments

Run this model

Scan QR code or launch in browser to test your prototype



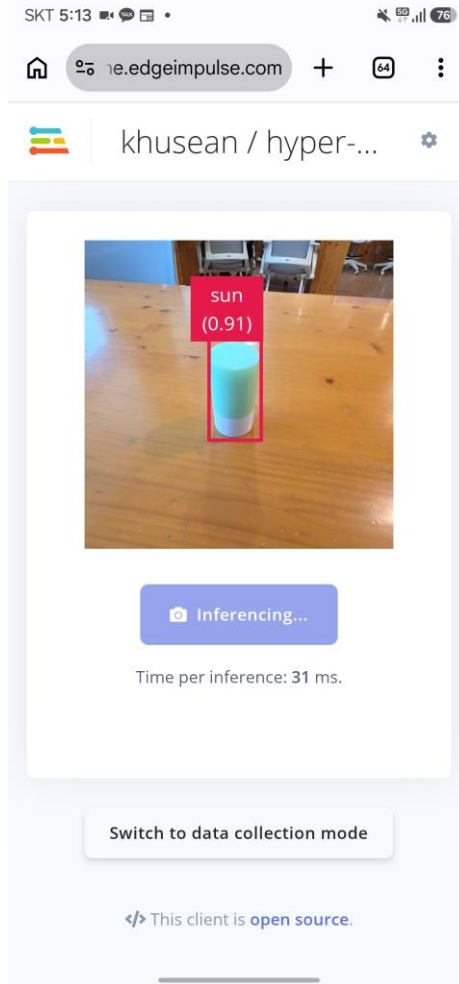
Launch in browser

Build output

tree ensemble_classifier.o -o build/linux-impulse-runner -lm -lstdc++ -l /

C++ library는 소스 코드 묶음입니다. 이 실습에서 Brick에 선택할 UNO Q 배포 대상으로 빌드합니다.

스마트폰 검출 결과 비교



Build 화면의 QR 코드로 간단히 확인

스마트폰으로 QR 코드를 열고 카메라에 팀 물체를 보여 줍니다.
박스 위치와 라벨이 물체에 맞는지 확인합니다.

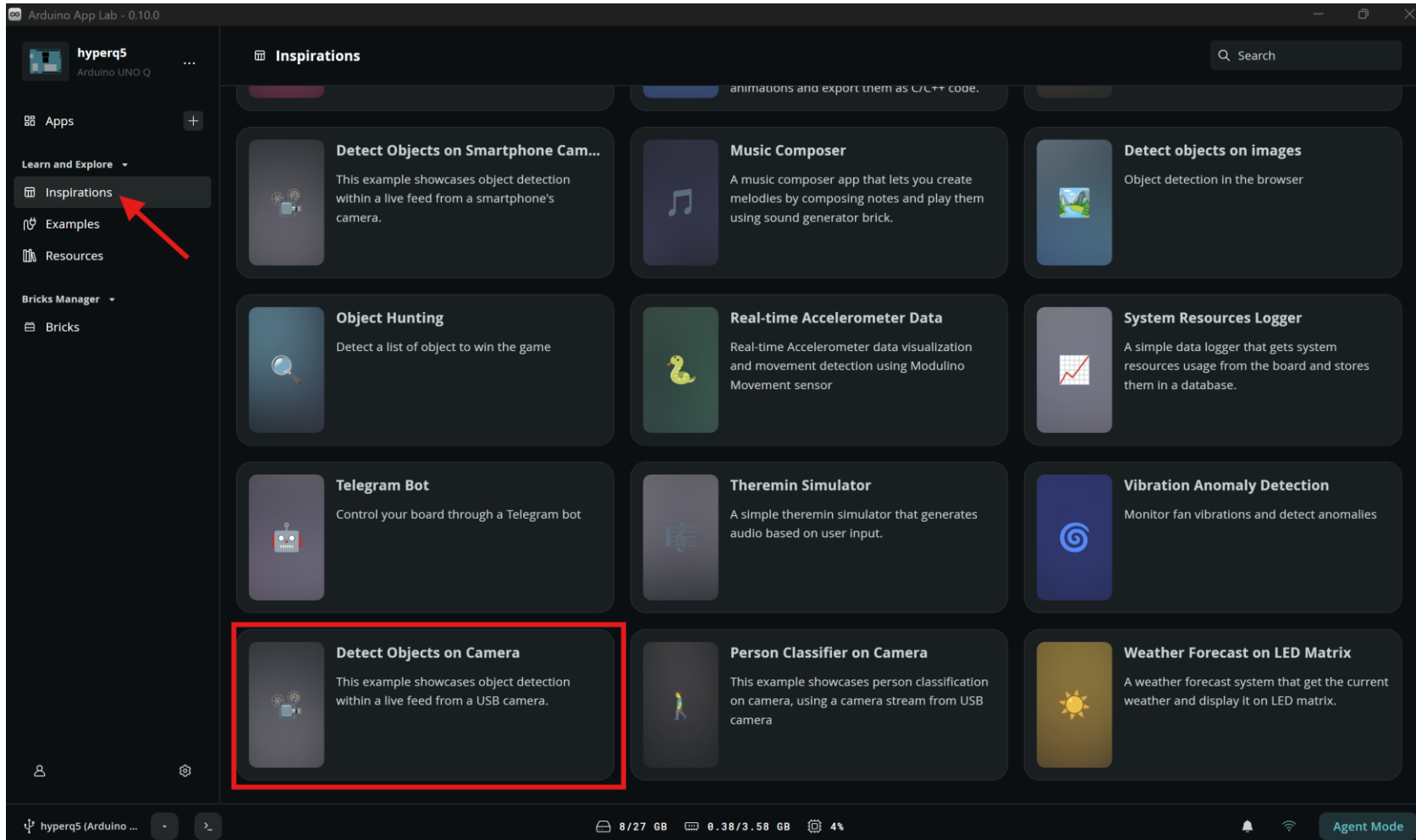
예시 화면

sun 0.91 검출, 추론 시간 31ms

스마트폰의 결과는 빠른 검출 확인용입니다.
UNO Q의 실제 FPS와 지연은 보드에서 따로 측정합니다.

팀 물체·배경·조명을 바꿔 확인합니다. 같은 숫자를 재현하는 것보다 라벨과 박스가 맞는지가 먼저입니다.

App Lab 검출 예제 찾기



확인할 것

Inspiration 목록의
Detect Objects
on Camera 선택

다음 화면에서
Copy and edit app
실행

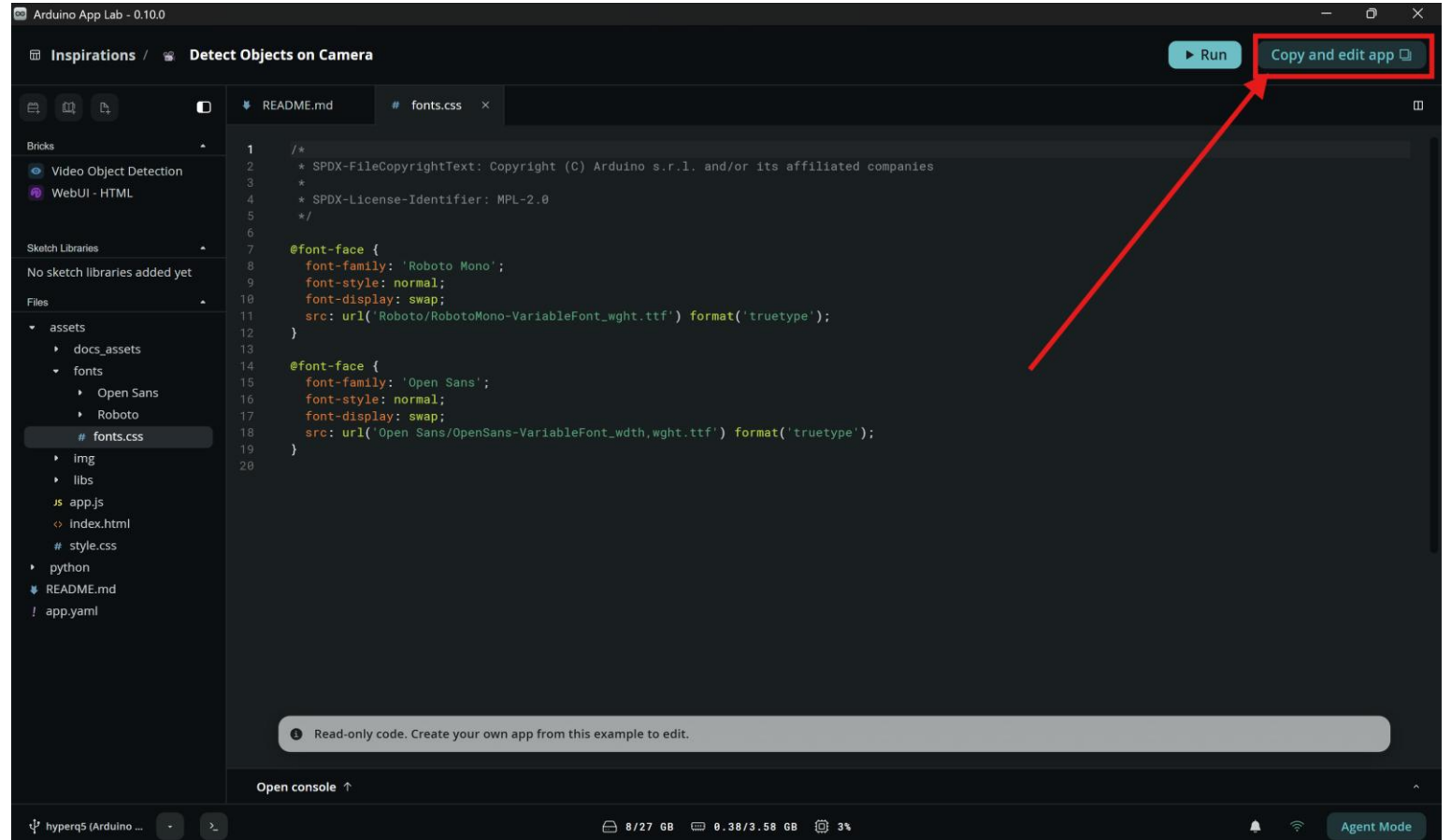
STEP 5에서는 모델 검출을 확인하고, STEP 7에서는 UI가 포함된 q-rover-03.zip을 임포트합니다.

App Lab 예제 복사

Detect Objects on Camera
예제 선택

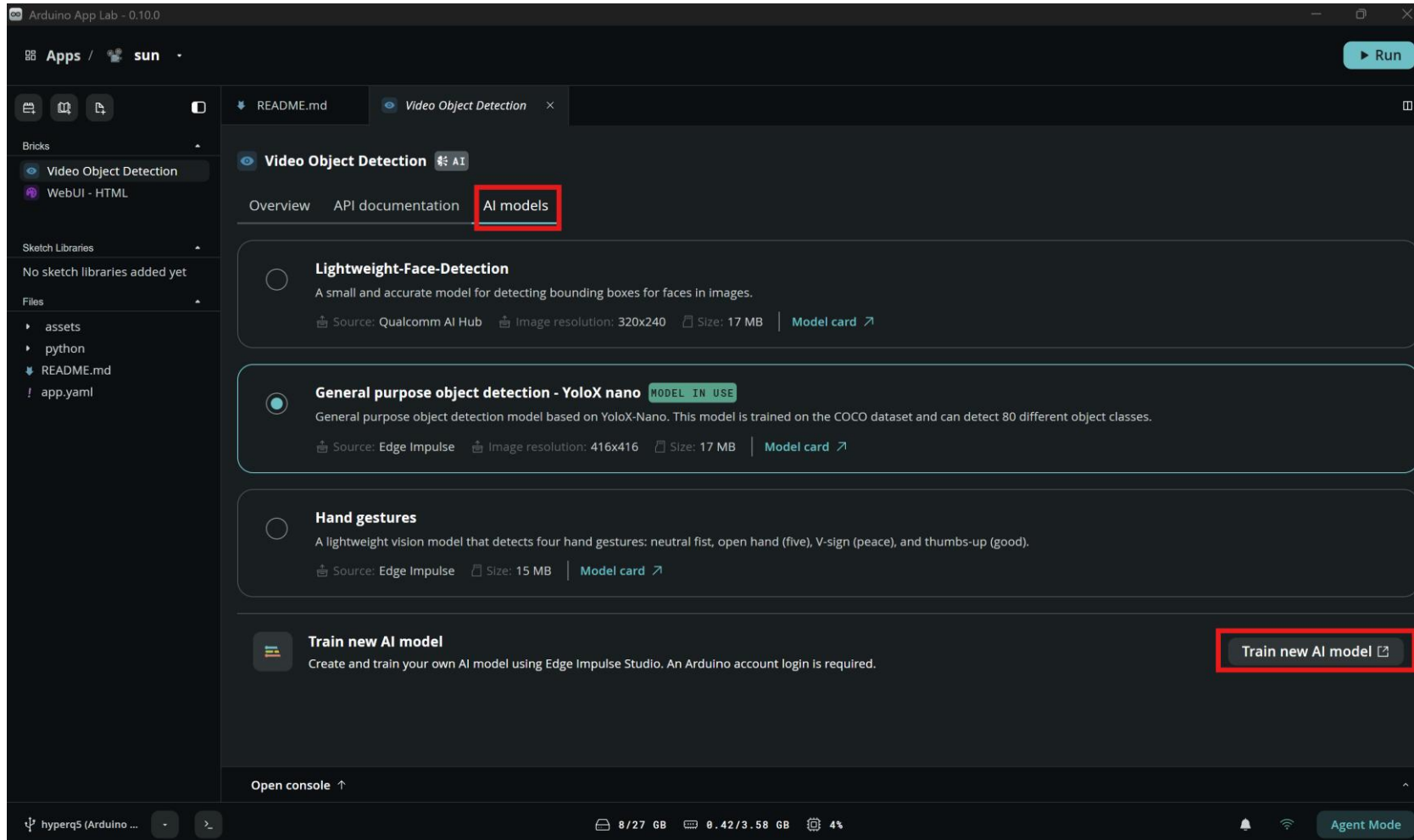
Copy and edit app
내 앱으로 복사

Video Object Detection
Brick 열기



이 단계에서는 우리 모델의 보드 검출을 먼저 확인합니다. 제어 앱 q-rover-03은 STEP 7에서 만듭니다.

Video Object Detection의 AI models



확인할 것

Video Object
Detection Brick 열기

AI models 탭 선택

우리 프로젝트의
Download 항목을
다음 화면에서 확인

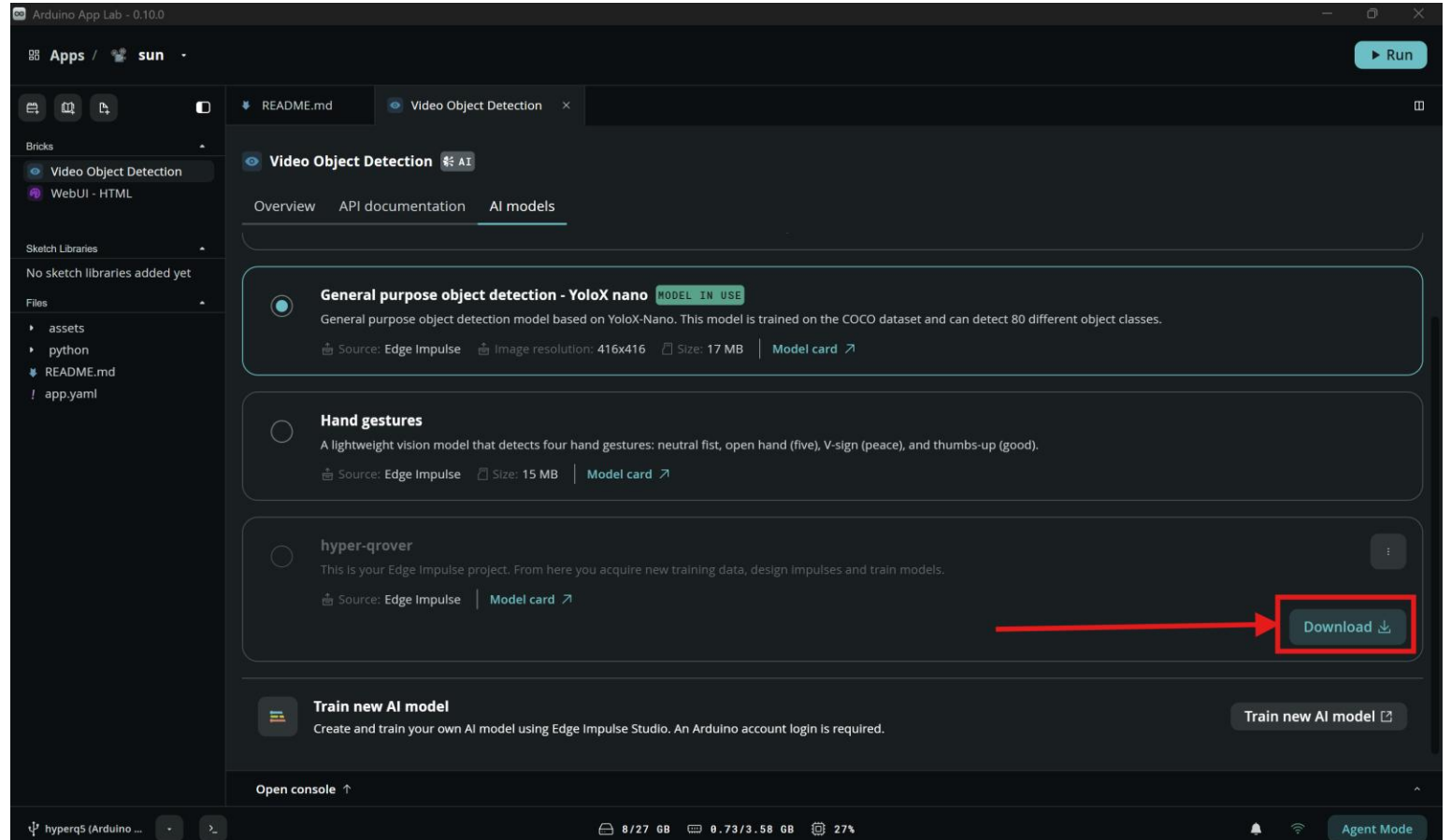
Train new AI model은 Edge Impulse로 이동하는 메뉴입니다. 준비한 팀 모델은 목록에서 Download합니다.

AI models 다운로드와 선택

AI models 탭에서
팀 프로젝트 Download

Brick Configuration
우리 모델 선택·Save

Run 후 보드 웹 화면
<http://<보드IP>:7000>



모델이 없으면 Target=UNO Q, 동일 이메일, OBJECT DETECTION, App Lab 버전을 먼저 확인합니다.

실행 가이드 ① 배터리 전환과 보드 IP

코드 반영 후 전원 연결 순서

1. 앱을 종료하고 유선 연결을 분리합니다.
2. 카메라를 허브에 꽂고, 보조배터리를 허브 전원 입력에 먼저 연결합니다.
3. 허브에 전원이 들어온 뒤, 허브를 UNO Q 보드에 연결합니다.

연결 순서가 바뀌면 카메라가 인식되지 않습니다.

현재 보드 IP 확인

유선 연결 후에는 설정해 둔 Wi-Fi와 관계없이 연결된 보드의 IP가 아래 ntfy 페이지에 표시됩니다. 팀 보드의 최신 알림을 확인합니다.

<https://ntfy.sh/hyperq-lab-0e1ce084dcc64a5c9d6d26135b28dec2>

주행 전 웹 화면과 카메라 검출을 확인합니다. 접속: `http://<보드 IP>:7000`

실행 가이드 ② CLI 실행·로그·종료

SSH 또는 App Lab 콘솔로 보드의 Linux 터미널에 접속해 실행합니다.

```
ssh arduino@<보드 IP>
```

<앱 이름>을 실제 앱 폴더 이름으로 바꿉니다. 꺾쇠 < >는 입력하지 않습니다.

앱 목록 `arduino-app-cli app list`

실행 `arduino-app-cli app start user:<앱 이름> -v`

로그 `arduino-app-cli app logs user:<앱 이름> --follow`

종료 `arduino-app-cli app stop user:<앱 이름>`

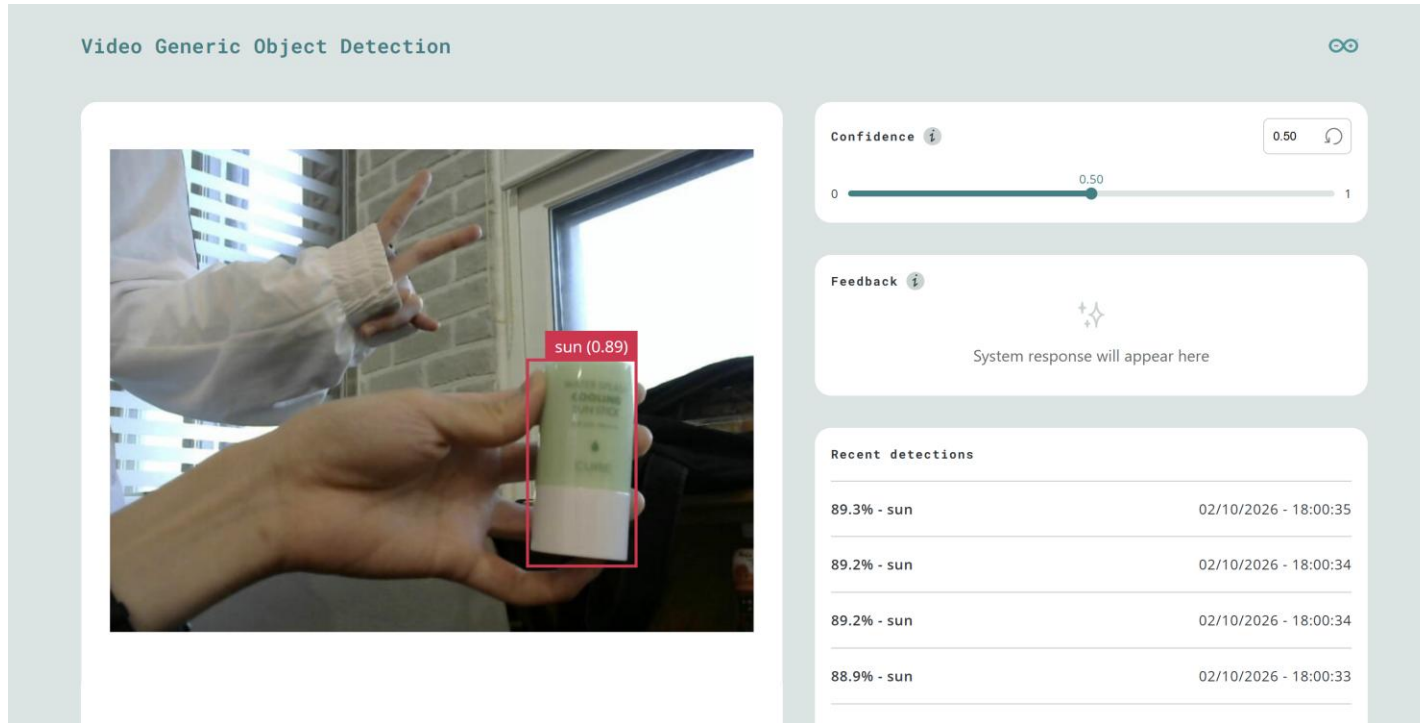
예시: 앱 이름이 q-rover-03일 때

```
arduino-app-cli app start user:q-rover-03 -v
```

```
arduino-app-cli app stop user:q-rover-03
```

로그 화면은 Ctrl+C로 빠져나옵니다. 앱 종료는 app stop 명령으로 합니다.

보드 검출과 좌표계 측정



카메라: 640×480
모델 입력: 416×416

물체를 오른쪽 끝에 놓고
x2 최댓값을 확인합니다.
박스 좌표의 기준 화면을 정합니다.

FRAME_W를 416 또는 640 근처의 실제 좌표 폭으로 정합니다. 표시 영상 폭과 반환 박스 좌표 폭을 혼동하지 않습니다.

로그와 보드 검출 속도

send_detections_to_ui 함수의 검출 반복 안에 추가

```
print(key, value.get("confidence"),  
      value.get("bounding_box_xyxy"), flush=True)
```

```
arduino-app-cli app logs user:<앱이름> --follow
```

목표를 계속 비추고 10초 동안 검출 콜백 수 / 10을 기록합니다.
한 프레임의 박스가 여러 개면 로그 줄 수를 그대로 세지 않도록 주의합니다.

임계값 비교와 체크포인트

오탐 박스가 한쪽에 뜨면 로버도 그쪽으로 꺾습니다. 오탐과 놓침을 함께 기록합니다.

임계값	물체 10번 중 검출	빈 화면 30초 오탐
0.3	팀 측정	팀 측정
0.5	팀 측정	팀 측정
0.7	팀 측정	팀 측정

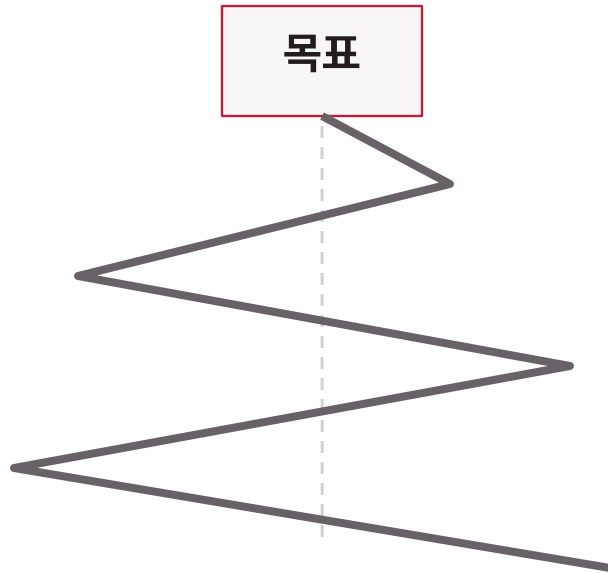
선택한 값을 main.py의 confidence에 적용합니다. 체크포인트: 보드 캡처·FRAME_W·콜백 FPS·임계값 비교.

Part 4

P제어: 목표를 화면 가운데로

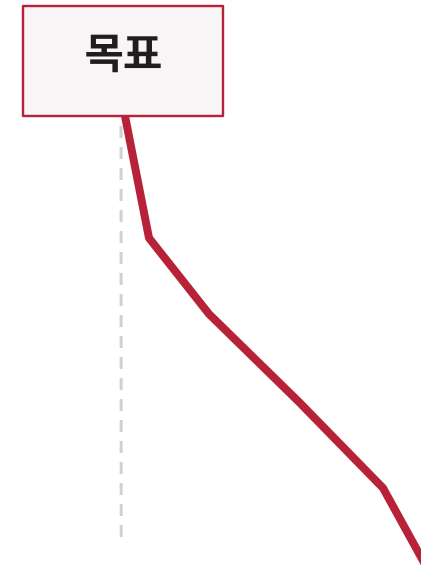
STEP 6 뱅뱅 제어와 P제어

뱅뱅 제어



조금 벗어나도 큰 조향
가운데를 지나 반대로 크게 조향

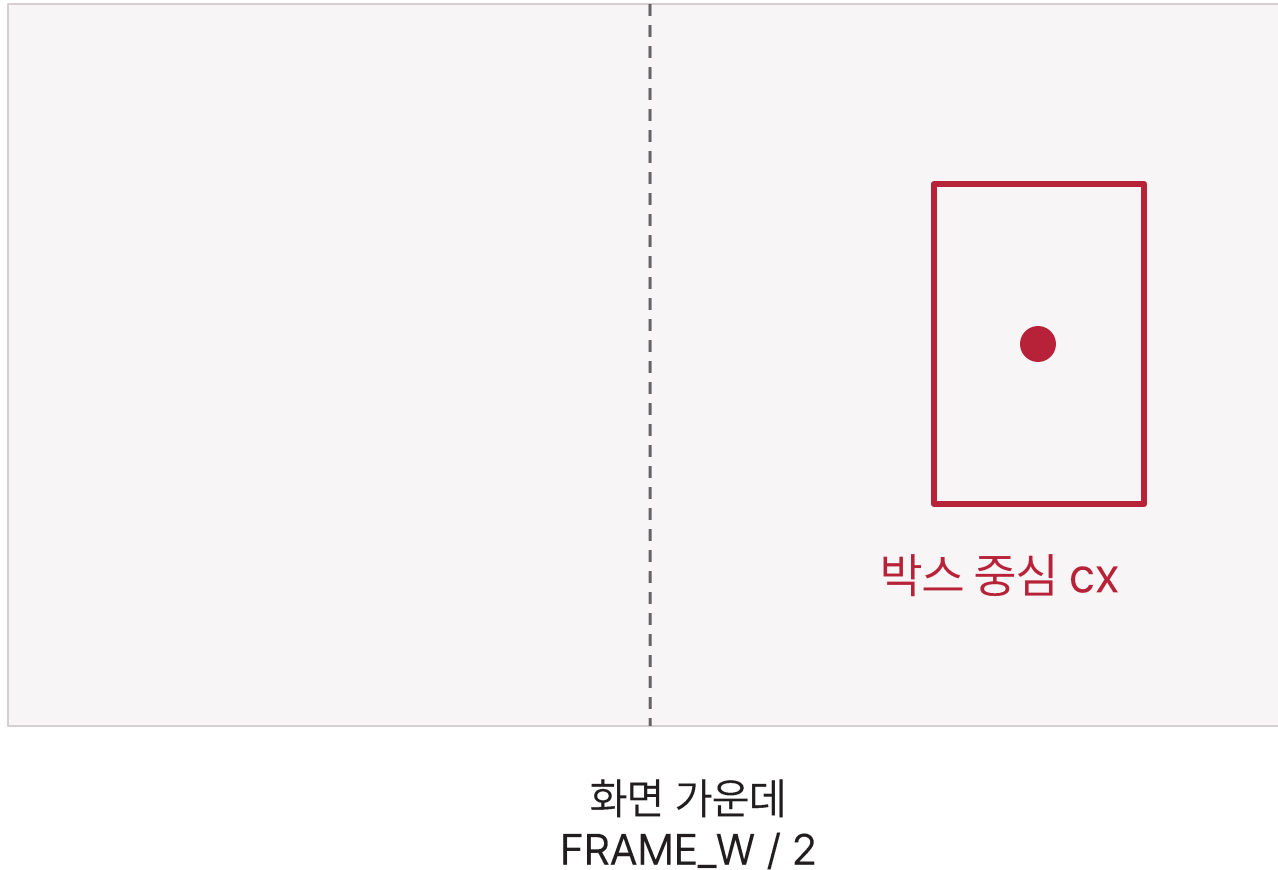
P제어



벗어난 만큼 비례해 조향
가운데에 가까울수록 조향 감소

경로는 원리 설명용 개념도입니다. 실제 로버의 경로·진동은 지연, 마찰, K_p 와 속도에 따라 달라집니다.

오차 계산: 박스 중심과 화면 중심



$$cx = (x1 + x2) / 2$$

가로 위치만 조향에 사용

오른쪽이면 +, 왼쪽이면 -

물체가 화면 가운데에 올 때 0입니다.
이 자료는 목표의 화면 위치를
오른쪽 양수로 정의합니다.

정규화: -100부터 +100

$$\text{err} = (\text{cx} - \text{FRAME_W}/2) / (\text{FRAME_W}/2) \times 100$$



해상도가 416에서 640으로 바뀌어도 같은 화면 비율이면 같은 오차입니다.

MCU와 공유하는 약속: 정수 -100~+100, +는 목표가 화면 오른쪽. 픽셀을 그대로 보내지 않습니다.

오차 계산 예시

입력·중간값	계산 결과
box	(300, 120, 380, 260)
FRAME_W	416
$cx = (300 + 380) / 2$	340
픽셀 오차 = $340 - 208$	132
정규화 = $132 / 208 \times 100$	약 +63

오른쪽 목표

+63을 MCU에 전달
양수 부호가 유지되어야
오른쪽으로 조향합니다.

정수 변환 방식에 따른 ± 1 은 허용합니다. FRAME_W 기준, 가운데 값 0, 부호를 먼저 확인합니다.

Kp와 조향 보정량

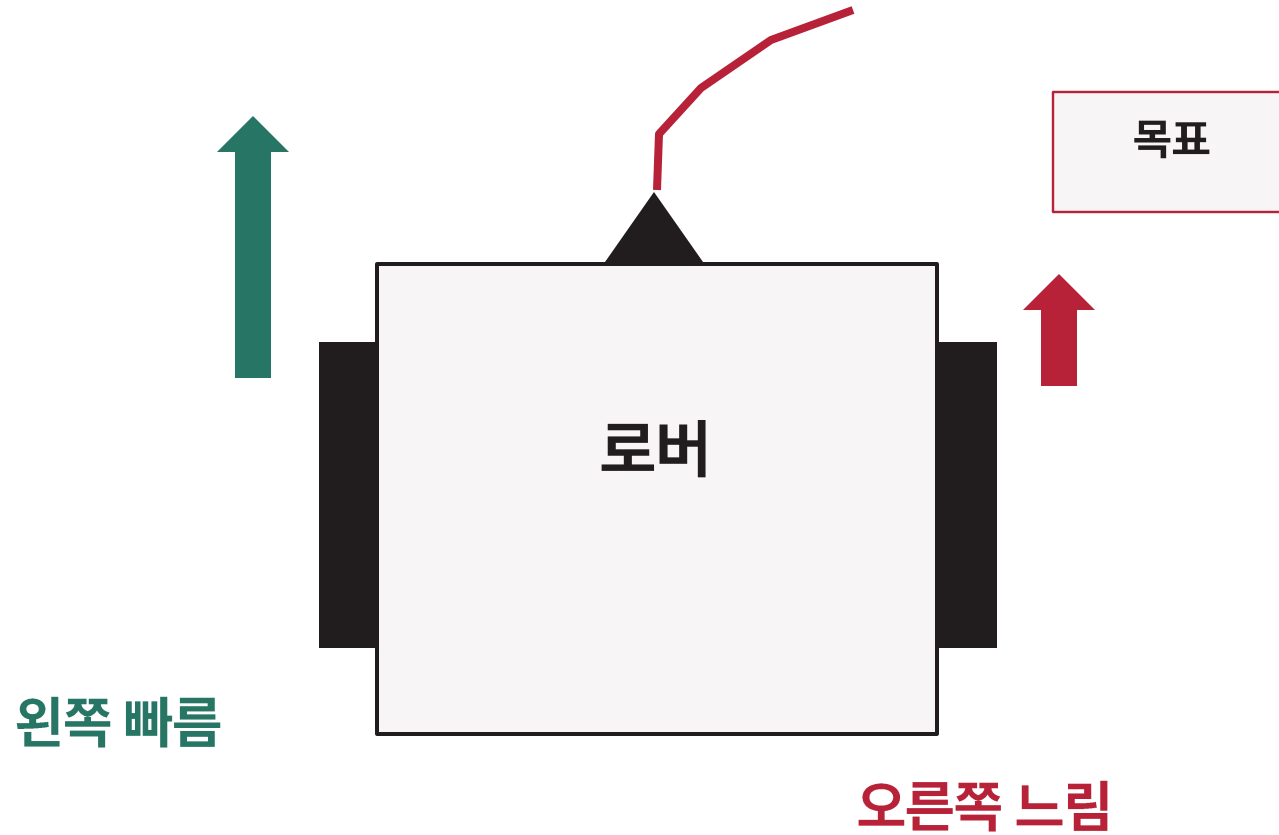
$$\text{turn} = \text{KP_STEER} \times \text{err}$$

err	Kp 0.1	Kp 0.2	Kp 0.4
10	1	2	4
50	5	10	20
100	10	20	40

Kp가 클수록
같은 오차에
더 크게 반응

turn은 각 바퀴에 더하거나 빼는 보정량입니다. 좌우 바퀴의 PWM 차이는 2×turn입니다.

차동 구동: 목표가 오른쪽일 때



$$\text{left} = \text{BASE_PWM} + \text{turn}$$

$$\text{right} = \text{BASE_PWM} - \text{turn}$$

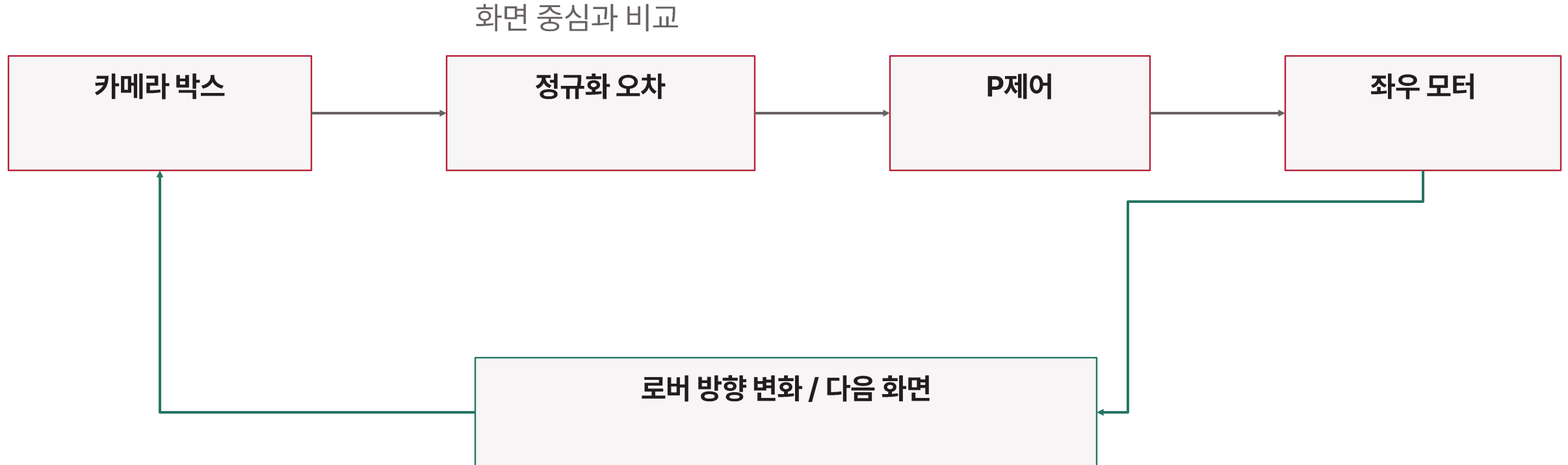
목표가 오른쪽이면 $\text{err} > 0$
왼쪽 바퀴가 더 빨라집니다.
로버가 오른쪽으로 휘며
물체가 화면 가운데로 옵니다.

부호 약속과 음의 피드백

목표 위치	err / turn	왼쪽 / 오른쪽	로버 방향
오른쪽	+ / +	빠름 / 느림	오른쪽으로 힘
가운데	0 / 0	같은 속도	직진
왼쪽	- / -	느림 / 빠름	왼쪽으로 힘

목표 쪽으로 돌수록 오차가 줄어야 합니다. 부호가 반대면 오차를 더 키우므로 바퀴를 들고 각 함수를 따로 확인합니다.

오차가 다시 입력으로 돌아오는 고리



이 흐름을 반복해 목표를 가운데로 맞춥니다. 목표가 사라지거나 명령이 오래되면 MCU가 정지합니다.

상한과 최소 PWM

turn을 $\pm\text{MAX_TURN}$ 안으로 제한

Kp가 커져도 전진 중 안쪽 바퀴가 역회전하지 않도록 조향량을 제한합니다.

예시	상한 전	상한 적용 후
err=100 / Kp=1.0 / MAX_TURN=40	turn=100	turn=40
BASE_PWM=90	left=190 / right=-10	left=130 / right=50

BASE_PWM – MAX_TURN > 측정한 직진 최소 PWM

자료의 90 / 0.2 / 40은 시작값입니다. 실제 바닥·배터리에서 측정한 최소 PWM에 맞춰 조정합니다.

지연된 오차와 지나침

촬영

MCU가 오차를 받음

다음 오차 도착

실제 +60

실제 오차는 이미
0을 지나 -25

오래된 +60으로
계속 조향

큰 Kp는 새 정보가 오기 전 더 많이 회전하게 만들어 흔들림을 키울 수 있습니다.

숫자는 지연 원리 설명용 가정입니다. 실제 검출 주기와 지연은 팀이 측정합니다. MCU 반복 주기와 구분합니다.

P제어 점검 질문

MPU는 인식, MCU는 바퀴와 정지

박스에서 오차까지만 계산해 보내면 카메라·모델이 바뀌어도 MCU의 입력 약속은 유지됩니다.
Python이 멈춰도 MCU의 watchdog과 비상정지가 동작해야 합니다.

두 함수의 부호를 모두 뒤집으면?

로버가 우연히 맞게 돌 수 있습니다. 한쪽만 바꾸면 문제가 드러납니다.
주행 방향만 보지 말고, 각 함수가 명세 표대로 값을 내는지 따로 확인합니다.

가운데 근처의 마찰과 남는 오차

제자리 회전은 작은 출력에서 멈출 수 있음

BASE_PWM=0에서 작은 turn은 정지마찰을 이기지 못할 수 있습니다.
최소 회전 PWM, 허용 오차, 조금 회전한 뒤 다음 프레임을 기다리는 방식이 필요합니다.

P제어만으로 오차가 항상 0이 되지는 않음

Kp가 작거나 좌우 모터 특성이 다르면 오차가 남을 수 있습니다.
이번 미션은 $|err| \leq 10$ 을 가운데로 봅니다. I·D 제어는 선택 과제로 비교합니다.

PART 4

STEP 7

기초 코드로 직접 구현

박스에서 오차, 오차에서 바퀴, 150mm 정지.

기초 코드를 먼저 실행하고 세 함수를 하나씩 확인합니다.

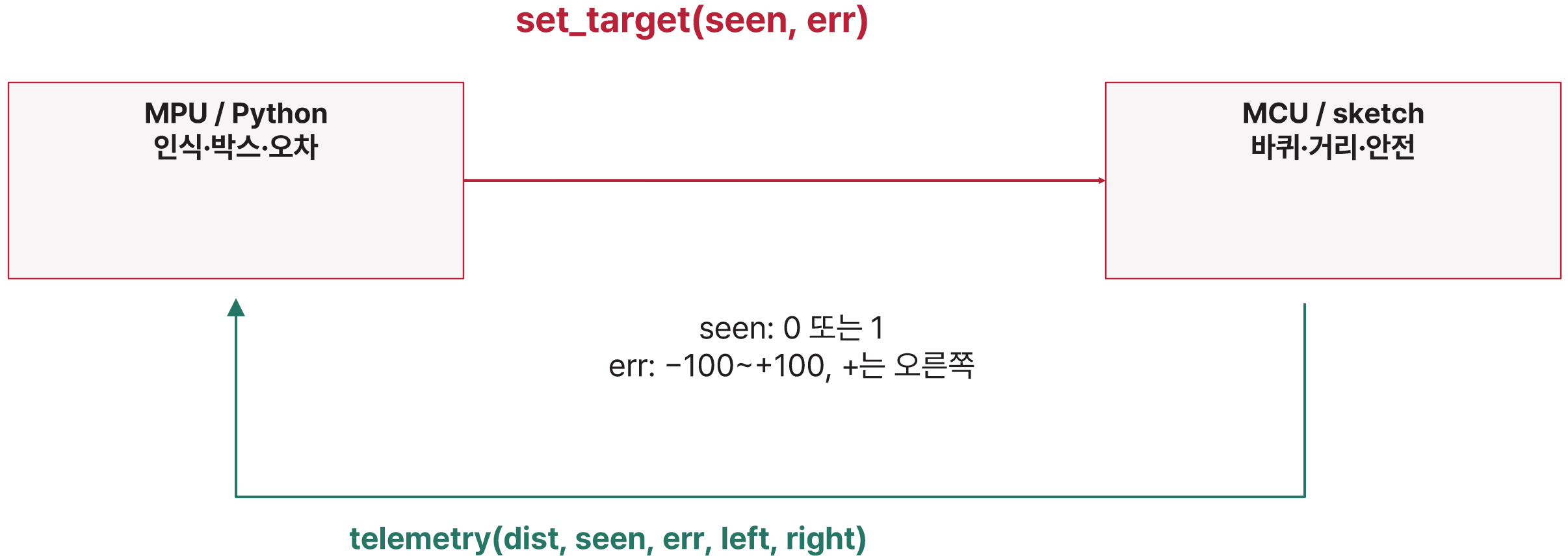
기초 코드와 팀 구현 범위

파일	제공된 기능	팀이 구현·설정
python/main.py	가장 큰 목표 박스 / HOLD / 3초 대기 heartbeat / BOX·TLM·FPS / 웹 화면	TARGET_LABEL / FRAME_W / confidence ① steer_error(box)
sketch/sketch.ino	모터 / ToF / 명령 받기 watchdog / 80mm 정지 / telemetry	② steerToWheels(err) ③ STOP_MM 정지 / 튜닝값
assets/	조향 오차·좌우 바퀴 표시	기본 화면 사용

미션 구현 후에는 바퀴를 들고 조향·정지 동작을 확인한 뒤 바닥 주행을 시작합니다.

기초 상태는 err=0, 바퀴=0입니다. 처음부터 움직이지 않는 것이 정상이며, BOX·TLM 로그부터 확인합니다.

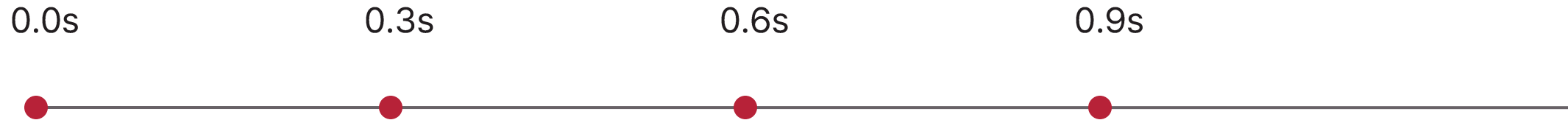
MPU와 MCU 사이의 약속



명령: 바뀔 때와 0.3초마다. 보고: 약 10Hz. 함수 이름, 인자 순서, 값의 범위를 양쪽에서 동일하게 유지합니다.

heartbeat와 watchdog

목표가 계속 가운데여도 명령을 주기적으로 보냅니다.



같은 (seen=1, err=0)도 다시 전송

MCU는 마지막 명령 이후 1초가 지나면 명령을 오래된 것으로 보고 정지합니다.

바뀔 때만 보내면 목표가 가만히 가운데에 있을 때 통신이 끊긴 것으로 판단할 수 있습니다.

App Lab 제어 앱 импорт

UI까지 포함된 q-rover-03 앱

1. 배포된 q-rover-03.zip을 App Lab의 Import로 불러옵니다.
2. sketch·python·assets·app.yaml과 Brick·라이브러리 설정이 함께 들어옵니다.
3. Video Object Detection의 AI models에서 팀 모델 Download
4. Brick Configuration에서 모델 선택·Save
5. TARGET_LABEL·FRAME_W 설정과 미션 구현 후 Run
6. <http://<보드 IP>:7000>에서 카메라·제어 UI 확인

Brick·라이브러리를 하나씩 추가할 필요가 없습니다.

선택: UI를 직접 만들고 싶다면 assets의 HTML·CSS·JS를 수정해도 됩니다.

코드 저장소: <https://github.com/emjdp/hyper-q-rover>

main.py의 처리 순서



콜백은 기록하고 끝내기

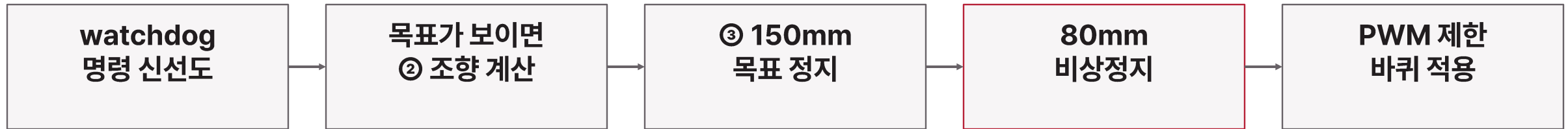
팀 라벨 박스 중 가장 큰 것을 선택합니다.
콜백 안에서 동기 Bridge.call을 하지 않습니다.

주기 루프가 MCU 명령 전송

시작 3초·HOLD_SEC 0.5초를 확인합니다.
보이면 steer_error()를 계산하고 전송합니다.

BOX는 박스·오차, TLM은 MCU 보고, FPS는 검출 콜백/초입니다. 실패 지점에 맞는 로그를 확인합니다.

sketch.ino의 정지 우선순위



비상정지가 앞에서 계산한 바퀴 값을 마지막에 덮어씹습니다.

Wheels는 left와 right를 함께 반환하는 구조체입니다.

set_target()은 수신값만 기록하고, setWheels()는 출발·정지 순간만 킥과 brake를 적용합니다.

미션 ① steer_error(box) 명세

```
def steer_error(box):  
    return 0    # 이 함수를 팀이 구현
```

box	FRAME_W	기대 err
(188, 100, 228, 200)	416	0
(300, 120, 380, 260)	416	63
(376, 50, 416, 150)	416	90
(0, 50, 40, 150)	416	-90
(440, 0, 520, 100)	640	50

박스 중심, 화면 중심 차이, 정규화, 정수 변환을 구현합니다. 반올림에 따른 ± 1 은 허용합니다.

미션 ② steerToWheels(err) 명세

BASE_PWM=90 / KP_STEER=0.2 / MAX_TURN=40 기준

err	왼쪽	오른쪽
0	90	90
50	100	80
-50	80	100
100	110	70
-100	70	110
100 / Kp=1.0	130	50

비례·방향·상한·전진을 구현해 Wheels {left, right}를 반환합니다. 마지막 행은 ± 40 포화를 확인하는 입력입니다.

미션 ③ 150mm 정지 명세

loop()의 미션 ③ 자리, 비상정지보다 위에 구현

유효한 거리에서 STOP_MM 아래로 내려가면 두 바퀴를 0으로 만듭니다.
거리 -1은 범위 밖 표시입니다. 가까운 물체로 처리하지 않습니다.

정지 후 목표를 뒤로 옮기면 다시 따라오기

목표가 계속 보이고 거리가 150mm 이상이면 접근을 다시 시작해야 합니다.
최종 시연에서는 줄자로 실제 $150 \pm 30\text{mm}$ 정지 거리를 확인합니다.

80mm 비상정지만으로 멈추면 미션 ③을 완료한 것이 아닙니다. 150mm 목표 정지를 별도로 구현합니다.

설정값을 정하는 위치

설정	파일	근거
TARGET_LABEL	main.py	labels.txt 정확히 동일
FRAME_W	main.py	STEP 5 오른쪽 끝 x2 측정
confidence	main.py	STEP 5 임계값 비교
BASE_PWM / MAX_TURN	sketch.ino	STEP 8 최소 PWM 측정
KP_STEER	sketch.ino	STEP 8 세 값 실험
STOP_MM	sketch.ino	미션 조건 150mm

값을 바꾼 이유와 실제 측정 조건을 README에 남깁니다. 카메라 해상도와 박스 좌표계 폭을 구분합니다.

AI 도구로 구현할 때의 검증

자료와 입력 약속을 함께 제공

STEP 6 개념, 통신 약속, 명세 표, 기초 코드 파일 전체를 제공하고
미션 표시한 곳을 채우도록 요청합니다.

명세 표로 손계산 후 바퀴를 들고 확인

입력값마다 기대 출력과 비교합니다. 특히 오차 부호와 포화를 확인합니다.
통신 함수 이름·인자·안전장치가 바뀌지 않았는지 검토하고, 각 줄의 이유를 기록합니다.

AI 도구 사용 여부보다 설명과 검증이 중요합니다. FRAME_W 측정값과 +가 오른쪽이라는 약속을 반드시 알려 줍니다.

구현 전과 미션 ① 확인

상태	확인	기대
기초 코드 그대로	TARGET_LABEL 적용 후 목표 비추기	BOX 줄 / err=0
5초 대기	FPS 로그	검출 콜백/초 표시
MCU 동작	TLM 로그	거리·seen·err·바퀴 보고
미션 ① 구현 후	목표 왼쪽 / 가운데 / 오른쪽	err 약 -90 / 0 / +90

웹 화면의 오차 막대와 BOX 로그가 같이 변해야 합니다. 이 단계에서는 바퀴 출력 구현을 아직 하지 않아도 됩니다.

바퀴를 들고 조향·안전 확인

확인	기대 동작
앱 시작 후 3초	바퀴 정지
목표 가운데	두 바퀴 같은 속도로 앞으로
목표 오른쪽	왼쪽 바퀴가 더 빠름
목표 왼쪽	오른쪽 바퀴가 더 빠름
목표 치우기	검출 지연과 HOLD 이후 약 1초 내외 정지
센서 앞 약 5cm에 손	정지
app stop	watchdog 기준 1초 이내 정지

체크포인트 6: 세 구현부 코드, err 세 값, 이 표의 결과. 이후 바닥에서 정면 1m 접근·15cm 정지를 확인합니다.

PART 4

STEP 8

튜닝과 Kp 비교

같은 출발 위치에서 Kp 세 값을 비교합니다.
로그로 지나침과 흔들림을 읽고, 선택 이유를 기록합니다.

최소 PWM 실험과 기본 속도

05-min-pwm/sketch.ino를 임시 앱에 적용

시연할 바닥에서 실행하고 arduino-app-cli monitor로 PWM을 봅니다.
두 바퀴가 다른 값에서 출발하면 큰 값을 기록합니다.

구간	측정 기준	기록
FWD	처음 직진하는 PWM + 5	직진 최소
SPIN	처음 제자리 회전하는 PWM + 5	회전 최소

BASE_PWM은 직진 최소 +30 근처에서 시작합니다. MAX_TURN은 BASE_PWM – 직진 최소보다 작게 정합니다.

Kp 세 값과 같은 실험 조건

1.5m 거리, 시작 err +70~+90. 물체·바퀴 위치를 테이프로 표시하고 Kp만 바꿉니다.

실험	KP_STEER	관찰할 내용
1	0.1	천천히 맞추는가 / 오차가 남는가
2	0.2	지나친 뒤 가운데로 돌아오는가
3	0.4	빠르지만 지그재그·놓침이 생기는가

0.2에서 이미 흔들리면 0.05·0.1·0.2, 0.4도 안정적이면 0.2·0.4·0.8을 비교합니다. 예측을 실측으로 확인합니다.

로그 수집과 노트북으로 가져오기

SSH에서 접근 한 번마다 CSV 저장

```
arduino-app-cli app logs user:q-rover-03 --follow  
| grep --line-buffered -o "TLM,.*" > kp02.csv
```

위 명령은 한 줄로 실행합니다. 끝나면 Ctrl+C. 파일명은 kp01 / kp02 / kp04로 구분합니다.

PowerShell에서 보드 파일 가져오기

```
scp arduino@<보드IP>:~/kp*.csv .
```

로그 구조와 그래프 생성

TLM,시각,seen,err,dist,left,right

목표를 처음 본 순간을 0초로 맞춰 세 실험을 겹칩니다.

```
pip install pandas matplotlib  
python plot_runs.py kp01.csv kp02.csv kp04.csv
```

상단: 조향 오차 / 하단: ToF 거리

회색 띠: 가운데 허용 범위 $|err| \leq 10$

Kp 그래프에서 읽을 지표

지표	뜻	판단
가운데까지(s)	$ err \leq 10$ 에 처음 들어간 시간	짧을수록 빠르게 정렬
반대쪽 최대	처음과 반대쪽으로 넘은 최대 오차	지나침 크기
좌우 바뀜	허용 범위 밖에서 부호가 바뀐 횟수	지그재그 횟수
마지막 오차	마지막 seen=1의 오차	가운데 정렬 여부
마지막 거리	마지막 유효한 ToF 거리	$150 \pm 30\text{mm}$ 확인

마지막 값만 보지 말고 로그·영상으로 실제 정지 시점을 확인합니다. 거리 -1은 그래프에서 제외합니다.

팀별 Kp 비교 기록표

실제 로버 실험값을 채웁니다. 이 표에는 시뮬레이션 값을 실측처럼 넣지 않습니다.

Kp	가운데(s)	반대쪽 최대	좌우 바뀜	마지막 err	정지(mm)
0.1	측정	측정	측정	측정	측정
0.2	측정	측정	측정	측정	측정
0.4	측정	측정	측정	측정	측정

체크포인트: approach.png, 비교표, 최소 PWM, BASE_PWM·MAX_TURN·Kp와 선택 이유 두 줄.

Part 5

시연과 제출

STEP 9 시연 합격 조건

조건	영상·기록에서 확인
① 대기·미검출 정지	시작 3초와 목표가 없는 동안 정지
② 조향 접근	정면 1.5m·옆 30cm 출발, 목표 쪽으로 방향을 틀며 접근
③ 목표 앞 정지	줄자로 150 ± 30mm 확인
④ 목표 제거	접근 중 목표를 치우면 약 1초 내외 정지
⑤ 안전 정지	전방 장애물 정지 / app stop 후 1초 안 정지
⑥ Kp 비교	Kp 세 값의 실측 그래프

학습 때의 조명 조건 중 하나에서 시연하고, 목표와 로버 사이에 다른 물체가 없도록 배치합니다.

주행 실패의 확인 순서

증상	먼저 확인
아예 안 움직임	시작 3초 / TARGET_LABEL / BOX / TLM seen / 미션 ②
목표 반대로 꺾음	오차·바퀴 부호 / 카메라 장착 방향
늘 오른쪽으로만 꺾음	화면 가운데를 빼서 가운데 err=0인지
가운데인데 err≠0	FRAME_W 재측정 (416과 640 혼동)
지그재그 / 안쪽 바퀴 정지	Kp·BASE_PWM / BASE_PWM-MAX_TURN과 최소 PWM
8cm에서야 정지	미션 ③ 150mm 정지 구현
목표 전부터 정지 / 가끔 정지	ToF 시야 / 거리 -1 / HOLD / 오탐 / FPS

실패 순간의 웹 화면과 TLM을 함께 기록합니다. 보이는 증상에서 시작해 값·조건·코드 순으로 좁힙니다.

제출 폴더와 큰 파일

팀 리포지토리 / week3
README.md / 라벨링규칙.md / labels.txt
train/ qrover_nano.py / train_log.txt / ap50.png
q-rover-03/ sketch / python / app.yaml
logs/ kp01.csv / kp02.csv / kp04.csv / approach.png
demo.mp4 합격 조건 ①~⑤가 보이는 1분 영상

데이터셋·.onnx·.pth는 Drive 링크

큰 파일을 리포지토리에 그대로 넣는 대신, README에서 저장 위치를 연결합니다.
수정한 코드와 측정표를 commit·push했는지 확인합니다.

README에 남길 측정값

구분	기록
데이터	클래스별 학습·검증 박스 / 세션 수 / 배경 장수 / 검증 세션
학습	전체 AP50 / AP50:95 / 클래스별 AP / 그래프
속도·보드	노트북 검출 계산 FPS / 보드 검출 콜백 FPS / FRAME_W / 임계값
제어	직진·회전 최소 PWM / BASE_PWM / MAX_TURN / KP_STEER
정지	실제 정지 거리 3회(mm)
실패	3개 사례의 증상 / 원인 / 해결

측정 조건과 단위를 함께 씁니다. FPS의 측정 범위가 다르면 직접 같은 속도 지표처럼 비교하지 않습니다.

구현 설명과 제출 전 확인

각 구현부를 설명

steer_error의 각 줄, steerToWheels의 각 줄과 부호 확인 방법,
미션 ③의 위치와 이유를 씁니다. AI를 썼다면 적용 위치와 검증 방법도 기록합니다.

최종 확인

미션 ①②③과 설정값 완료 / 합격 조건이 영상에 보임
Kp 그래프와 선택 이유 완료 / README의 측정표·실패 사례 완료
팀 리포지토리에 push 완료

선택 확장: 거리와 탐색

확장	핵심 개념	함정
거리도 P제어	속도를 $Kp \times (\text{거리} - 150)$ 로 계산	최소 PWM / 거리 -1 / 정지 허용 범위
제자리 탐색·정렬	목표를 찾고 가운데 맞춘 뒤 접근	회전 최소 PWM / 허용 오차 / 프레임 기다리기
박스 크기로 거리	박스 높이 오차로 속도 제어	실물 크기·자세에 따라 크기 변화
표지판 따르기	새 라벨 / 상태 머신	연속 검출 / 같은 표지판 재처리 쿨다운

공통 미션을 완료한 뒤 팀이 설계합니다. 추가 클래스는 촬영·라벨링·학습·배포를 다시 진행합니다.

선택 확장: I와 D

I는 남은 오차를 줄이는 데, D는 빠른 변화에 반응해 흔들림을 줄이는 데 사용할 수 있습니다.

항	계산	필요한 처리
P	현재 오차에 비례	Kp / 포화 / 부호 검증
I	오차를 시간에 따라 누적	누적 상한 / 목표 놓치면 초기화
D	오차 변화 / 경과 시간	새 프레임이 도착했을 때만 갱신

추가 항은 자동으로 성능을 보장하지 않습니다. 같은 조건에서 P 기준과 로그·정지·진동을 비교합니다.

Part 6

11/2 발표와 팀 프로젝트

공통 미션 결과 발표 6~8장

밑의 표는 발표 내용 예시입니다. 동일하게 구성할 필요는 없으며, 분량은 6-8장으로 제한합니다.

장	내용
1	팀 물체와 데이터 조건
2	학습 결과와 AP50 그래프
3	같은 모델의 노트북·보드 실행
4	우리가 짠 P제어와 부호 검증
5	Kp 실측 그래프와 선택 이유
6	1분 시연 영상
7~8	가장 오래 막힌 실패 / 선택 확장

공통 미션 결과와 팀 프로젝트 계획을 이어 발표합니다. 팀당 발표·피드백 시간은 추후 운영진이 공개합니다.

팀 프로젝트 계획서 4~5장

장	내용	구체적으로 적을 것
1	무엇을	누가·어떤 상황에서·무엇을 하는 로봇인지 한 문장과 그림
2	어떻게	공통 미션에서 재사용할 것 / 새 기술·부품
3	언제까지	연말 발표회까지 마일스톤 3~4개
4	위험	안 될 수 있는 것 1~2개 / 대안
5	피드백 요청	시니어에게 묻고 싶은 것

하드웨어 추가 시 부품·예산을 함께 적습니다. 발표 이후 피드백을 반영해 계획서를 제출합니다.

팀 프로젝트에선 Q-Rover를 사용해도 되고, 새로운 로봇을 사용해도 됩니다. 혹은 로봇을 사용하지 않고 simulation만 사용해도 되며 주제는 로봇/비전과 관련된 주제인 한 자율입니다.

팀 프로젝트 응용 예시 ① 비전·미션·음성

분야와 과제 예시	재사용하는 것	새로 필요한 것
A · 비전 심화 (AMR 계열) 사람·공·ArUco 마커 추종	박스 중심 조향·P제어 ToF 정지·안전 제어	검출한 목표와 일정 거리 유지 박스 크기·마커 자세로 거리 추정 응용: OpenCV 라인 따라가기
B · 비전+미션 (AGV 계열) 지정 지점으로 물건 운반·하역	MCU 명령·상태 전환 조향·정지 제어	ArUco·색상·QR 마커로 경로점 인식 서보로 바구니 입구를 막아 물건 적재 도착 지점에서 입구를 열어 내려놓기
C · 음성 “이리 와”, “멈춰” 명령	통신·모터 제어 정지 우선순위	온디바이스 Vosk 또는 Whisper API C270i 내장 마이크로 음성 입력 3m 거리에서 명령 인식을 확인 응용: 수신호 인식·음성 조종

입력과 행동을 확장하되 공통 미션의 조향·정지·통신 구조를 재사용합니다.

팀 프로젝트 응용 예시 ② 에이전트·제어·자체 기획

분야와 과제 예시	재사용하는 것	새로 필요한 것
D · LLM 에이전트 주차장 빈자리 찾아가기	검출·거리·상태 요약 MCU 통신·안전 정지	LLM이 센서 요약을 읽고 이동 계획 생성 계획을 로버 명령과 상태 전환으로 실행 응용: 순찰 중 사진·ntfy 알림
E · 정밀 제어 엔코더 모터·정밀 주차	로그·PWM 구동·P제어 Kp 비교 실험	엔코더 포함 모터로 업그레이드 페루프 속도 제어·PID 튜닝 주행·주차 오차를 측정해 비교
F · 자체 기획 위 다섯 분야 밖의 주제	재사용할 코드·센서·부품 새로 필요한 기술 구분	목표와 구현 방법을 멘토에게 제안 멘토 승인 후 진행 시연 기준·일정·예산을 구체화

D 참고: UNO Q LLM 로봇 제작 사례 (Hackster)

E 부품 후보: 엔코더 모터 (디바이스마트)

추가 확장: 모델 비교·ROS 2도 계획서에 재사용 범위와 새 기술을 적어 제안할 수 있습니다.

부록

문제 해결과 참고 자료

촬영·라벨링·학습 문제 해결

증상	확인
카메라 검은 화면	--cam 번호 / 다른 앱의 C270 사용 여부
카메라·보드 미인식	허브 전원 먼저, 그다음 보드 연결
Windows 한글 경로에서 이미지 실패	제공 코드의 imencode·tofile / imdecode·fromfile 경로 사용
세션이 1개뿐이라는 오류	s01_0000.jpg 형식 / _ 앞의 세션 이름
AP가 0에 가까움	labels.txt 이름·순서 / 각 txt 클래스 번호 / 라벨 누락
Colab 중단	Drive 저장 여부 / GPU / 위에서부터 셀 순서

질문할 때 오류 메시지, 실행한 STEP, 관련 파일, 재현 조건, 로그 일부를 함께 보냅니다.

배포·통신 문제 해결

증상	확인
티 박스가 이상함	YOLOX / Squash / 0..255 / labels.txt
App Lab 모델 목록에 없음	Target UNO Q / 계정 이메일 / 카테고리 / App Lab 버전
보드에서만 약한 검출	RGB·크기·값 범위 / 촬영 시점·조명
BOX 로그가 없음	TARGET_LABEL 철자 / Brick Configuration의 모델
TLM 로그가 없음	telemetry 문자열 / 인자 5개 / 스케치 업로드
CSV가 비어 있음	TLM 출력 / --line-buffered 옵션
주행 중 보드가 꺼짐	보조배터리 5V 3A 출력·잔량 / 허브 연결

보드 IP는 ntfy의 팀 보드 최신 알림에서 확인하고, 해당 주소로 다시 접속합니다.

ROS 2로 이어지는 개념

역할을 옮기는 개념 대응표입니다. Bridge와 ROS 통신 방식이 그대로 호환되는 것은 아닙니다.

Q-ROVER 구조	ROS 2에서의 대응
검출 결과 콜백	vision_msgs/Detection2DArray 구독
ONNX detect()	카메라를 구독하는 검출 노드
박스에서 조향 오차	목표 추종 노드
set_target / 좌우 바퀴	cmd_vel / 하위 컨트롤러
watchdog·비상정지	명령 타임아웃·안전 제어
웹 UI·TLM·그래프	rviz2·rosbag·PlotJuggler 등

현재의 전처리·후처리와 부호 약속을 유지하고, ROS 메시지 형식·좌표·시간 기준에 맞춰 연결합니다.